

DevOps e integración continua

Breve descripción:

Este componente formativo aborda los fundamentos de DevOps, los procesos de desarrollo de software, el uso de contenedores, la integración, entrega y despliegue continuo, así como las herramientas Git, GitHub, GitLab y Jenkins. Estos conocimientos permiten comprender la automatización y la integración continua como soporte para el desarrollo y la entrega de software.

Tabla de contenido

Introducción	5
1. Introducción a DevOps	8
1.1. Historia y evolución de DevOps	9
1.2. Principios de DevOps	11
1.3. Cultura de colaboración y automatización	12
1.4. Beneficios de la implementación de DevOps	14
2. Procesos de desarrollo de software	17
2.1. Ciclo de vida del desarrollo de software	18
2.2. Actividades del proceso de desarrollo	19
2.3. Roles y responsabilidades del equipo de desarrollo	21
2.4. Artefactos de software	23
2.5. Herramientas para el desarrollo de software	25
3. Contenedores en entornos de desarrollo	28
3.1. Características de los contenedores	28
3.2. Beneficios del uso de contenedores	30
3.3. Aplicaciones de los contenedores en DevOps	32
3.4. Uso de contenedores en entornos DevOps	33
4. Integración, entrega y despliegue continuo (CI/CD)	36

4.1. Integración continua (CI)	36
4.2. Entrega continua (CD).....	38
4.3. Despliegue continuo	40
4.4. Beneficios de la automatización de procesos CI/CD.....	42
4.5. Flujo de trabajo en entornos DevOps.....	43
5. Gestión de versiones con Git	46
5.1. Fundamentos del control de versiones	46
5.2. Git como soporte para la integración continua	48
5.3. Repositorios remotos con GitHub y GitLab	51
5.4. Estrategia de ramificación GitFlow	55
5.5. Buenas prácticas en la gestión de código fuente.....	59
6. Automatización de procesos CI/CD	62
6.1. Introducción a Jenkins.....	62
6.2. Arquitectura y componentes de Jenkins	65
6.3. Creación de pipelines de integración continua	68
6.4. Automatización de procesos de construcción, pruebas y despliegue	71
6.5. Monitoreo y gestión de procesos CI/CD	74
Síntesis	77
Glosario	78

Referencias bibliográficas	80
Créditos	82

Introducción

En la actualidad, las organizaciones requieren desarrollar y entregar soluciones de software con mayor rapidez, calidad y capacidad de adaptación a los cambios del entorno. Para responder a estas necesidades, DevOps integra prácticas, procesos y herramientas que fortalecen la colaboración entre los equipos de desarrollo y operaciones, promoviendo la automatización y la mejora continua durante el ciclo de vida del software.

Este componente formativo presenta los fundamentos de DevOps, los procesos de desarrollo de software y el papel de los contenedores como tecnología para la creación de entornos consistentes y escalables. Asimismo, aborda los conceptos de integración, entrega y despliegue continuo (CI/CD), destacando su contribución a la automatización de las actividades de construcción, prueba y publicación de aplicaciones.

Finalmente, se desarrollan los principios del control de versiones mediante Git, el uso de repositorios remotos con GitHub y GitLab, la estrategia de ramificación GitFlow y la automatización de procesos con Jenkins. Estos conocimientos proporcionan una base para comprender cómo las prácticas y herramientas DevOps favorecen el trabajo colaborativo, optimizan los procesos de desarrollo y contribuyen a la entrega continua de soluciones de software con mayor calidad y confiabilidad.

Para comprender la importancia del contenido y los temas abordados, se recomienda acceder al siguiente video:

Video 1. DevOps e integración continua



[Enlace de reproducción del video](#)

Transcripción del video. DevOps e integración continua

Cada día, millones de personas utilizan aplicaciones para estudiar, trabajar, comprar o comunicarse. Sin embargo, detrás de cada actualización, cada nueva función y cada mejora existe un equipo de profesionales que trabaja de manera coordinada para desarrollar, probar e implementar software de forma rápida y confiable.

Hace algunos años, estos procesos se realizaban por separado. Mientras el equipo de desarrollo creaba nuevas funcionalidades, el equipo de operaciones se encargaba de ponerlas en funcionamiento. La falta de comunicación entre ambos generaba retrasos, errores y dificultades para entregar aplicaciones de calidad.

Transcripción del video. DevOps e integración continua

Con el tiempo surgió una nueva forma de trabajar: DevOps, término que proviene de la unión de las palabras Development (desarrollo) y Operations (operaciones). Este enfoque promueve la colaboración, la automatización y la integración continua para acelerar el desarrollo de software sin perder estabilidad ni calidad.

Con esta nueva forma de trabajar, los equipos comenzaron a incorporar tecnologías y metodologías que facilitaron la colaboración, la automatización de tareas y la entrega continua de software. Estas prácticas transformaron la manera de desarrollar aplicaciones, haciendo posible crear soluciones más confiables, eficientes y adaptadas a las necesidades de los usuarios.

Lo que comenzó como una necesidad de mejorar la comunicación entre los equipos evolucionó hasta convertirse en una nueva forma de desarrollar software, donde la colaboración, la automatización y la integración continua permiten transformar ideas en soluciones tecnológicas de manera más ágil, segura y eficiente.

1. Introducción a DevOps

El desarrollo de software ha evolucionado para responder a las necesidades de organizaciones que requieren entregar aplicaciones de manera más rápida, confiable y alineada con las expectativas de los usuarios. En este contexto, DevOps surge como un enfoque que integra personas, procesos y herramientas con el propósito de fortalecer la colaboración entre los equipos de desarrollo y operaciones, optimizando el ciclo de vida del software desde la planificación hasta el despliegue y mantenimiento.

El término DevOps proviene de la unión de las palabras Development (desarrollo) y Operations (operaciones). Más que una tecnología o una herramienta específica, representa una cultura de trabajo basada en la comunicación, la colaboración y la automatización de actividades, con el fin de mejorar la calidad del software y acelerar su entrega. Este enfoque promueve la participación conjunta de los diferentes equipos involucrados en el desarrollo, favoreciendo la detección temprana de errores, la integración continua de cambios y la entrega frecuente de nuevas funcionalidades.

DevOps también impulsa la adopción de prácticas que permiten automatizar tareas repetitivas, estandarizar procesos y fortalecer la retroalimentación durante todas las etapas del desarrollo. Para ello, se apoya en metodologías, herramientas y tecnologías que facilitan la integración del código, la ejecución de pruebas, el despliegue de aplicaciones y el monitoreo de los servicios en funcionamiento.

En la actualidad, DevOps es ampliamente utilizado en proyectos de desarrollo de software y en entornos de computación en la nube, donde la rapidez de respuesta, la escalabilidad y la disponibilidad de los servicios son factores determinantes. Su implementación favorece equipos de trabajo más colaborativos, procesos más eficientes y una mayor capacidad para adaptarse a los cambios del entorno tecnológico.

1.1. Historia y evolución de DevOps

Antes de la adopción de DevOps, los equipos de desarrollo y los equipos de operaciones trabajaban de forma independiente. Mientras los desarrolladores se enfocaban en crear nuevas funcionalidades para el software, el personal de operaciones era responsable de implementarlo, administrarlo y garantizar su disponibilidad. Esta separación generaba dificultades de comunicación, retrasos en las entregas, errores durante los despliegues y una limitada capacidad para responder a los cambios del entorno.

Con el crecimiento de las aplicaciones web, los servicios en la nube y la necesidad de realizar actualizaciones con mayor frecuencia, las organizaciones comenzaron a buscar nuevas formas de mejorar la colaboración entre ambos equipos. En este contexto surgió DevOps, un enfoque que promueve la integración de personas, procesos y herramientas para optimizar el desarrollo, la implementación y el mantenimiento del software.

Desde su aparición a finales de la primera década del siglo XXI, DevOps ha evolucionado mediante la incorporación de prácticas orientadas a la automatización, la integración continua, la entrega continua y el monitoreo permanente de las aplicaciones. Esta evolución también ha estado acompañada por el uso de tecnologías como los contenedores, la computación en la nube y las plataformas de automatización, que facilitan la creación de entornos más flexibles, escalables y confiables.

En la actualidad, DevOps constituye una práctica ampliamente adoptada por organizaciones de diferentes sectores, ya que favorece ciclos de desarrollo más ágiles,

mejora la calidad del software y fortalece la capacidad de responder de manera oportuna a las necesidades de los usuarios y del negocio.

Figura 1. Evolución del desarrollo tradicional hacia el enfoque DevOps



- **Desarrollo tradicional**
 - Equipos separados
- **Necesidad de mejorar la colaboración**
 - Surgimiento de DevOps
 - Integración de desarrollo y operaciones
- **Automatización de procesos**
 - CI/CD
 - Contenedores
 - Nube
- **DevOps en la actualidad**
 - Mayor colaboración

- Entregas continuas
- Mejor calidad
- Adaptación al cambio

1.2. Principios de DevOps

DevOps se fundamenta en un conjunto de principios que orientan la forma en que las organizaciones desarrollan, implementan y mantienen aplicaciones de software. Estos principios promueven la integración entre las personas, los procesos y las herramientas, con el propósito de mejorar la calidad del software, optimizar los tiempos de entrega y fortalecer la colaboración durante todo el ciclo de vida del desarrollo.

Entre los principios que sustentan el enfoque DevOps se encuentran:

Figura 2. Principios fundamentales de DevOps



- **Colaboración:** promueve el trabajo conjunto entre los equipos de desarrollo, operaciones y demás áreas involucradas, favoreciendo la comunicación y la responsabilidad compartida.
- **Automatización:** permite ejecutar de forma automática tareas relacionadas con la construcción, las pruebas, el despliegue y el monitoreo de las aplicaciones, reduciendo errores y aumentando la eficiencia.
- **Integración continua:** facilita la incorporación frecuente de cambios al código fuente y su validación mediante pruebas automatizadas.
- **Retroalimentación continua:** proporciona información permanente sobre el comportamiento de las aplicaciones para identificar oportunidades de mejora y responder oportunamente a los incidentes.
- **Mejora continua:** impulsa la evaluación y optimización constante de los procesos, con el fin de incrementar la calidad del software y la eficiencia del trabajo colaborativo.

Estos principios se complementan entre sí y constituyen la base para implementar prácticas DevOps orientadas a la colaboración, la automatización y la entrega continua de soluciones de software.

1.3. Cultura de colaboración y automatización

La cultura DevOps promueve una forma de trabajo basada en la colaboración, la comunicación y la responsabilidad compartida entre los equipos que participan en el desarrollo y la operación del software. Este enfoque busca eliminar las barreras

tradicionales entre las diferentes áreas, favoreciendo el intercambio de conocimientos y la coordinación de actividades durante todo el ciclo de vida de las aplicaciones.

En este contexto, desarrolladores, personal de operaciones, especialistas en calidad y demás integrantes del equipo trabajan de manera conjunta para planificar, desarrollar, probar, implementar y monitorear las soluciones de software. Esta colaboración permite identificar y resolver inconvenientes de forma temprana, mejorar la calidad de los productos y responder con mayor rapidez a las necesidades de los usuarios.

Otro elemento esencial de la cultura DevOps es la automatización, la cual consiste en utilizar herramientas que ejecutan de forma automática tareas repetitivas del proceso de desarrollo. La automatización reduce la intervención manual en actividades como la compilación del código, la ejecución de pruebas, la integración de cambios, el despliegue de aplicaciones y el monitoreo de los servicios, contribuyendo a disminuir errores y optimizar los tiempos de trabajo.

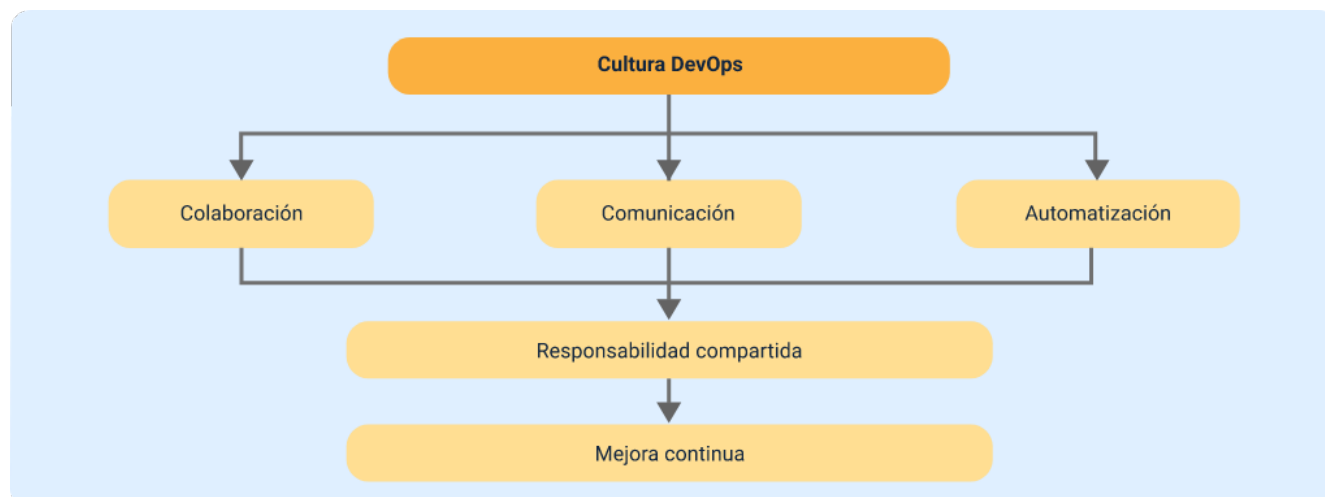
Las principales características de una cultura DevOps son:

- **Trabajo colaborativo:** promueve la participación activa de todos los integrantes del equipo durante el ciclo de vida del software.
- **Comunicación permanente:** facilita el intercambio continuo de información entre las áreas involucradas.
- **Responsabilidad compartida:** fomenta el compromiso conjunto con la calidad, la disponibilidad y el funcionamiento de las aplicaciones.
- **Automatización de procesos:** disminuye la ejecución manual de tareas repetitivas y mejora la eficiencia.

- **Mejora continua:** impulsa la evaluación constante de los procesos para identificar oportunidades de optimización.

La cultura de colaboración y automatización constituye uno de los pilares de DevOps, ya que favorece la integración entre las personas, la optimización de los procesos y el uso de herramientas que contribuyen al desarrollo continuo de software. A continuación, se resumen los principales elementos que caracterizan este enfoque de trabajo.

Figura 3. Cultura de colaboración y automatización en DevOps



1.4. Beneficios de la implementación de DevOps

La implementación de DevOps aporta beneficios que favorecen el desarrollo, la entrega y el mantenimiento de aplicaciones de software. La integración entre las personas, los procesos y las herramientas permite optimizar las actividades del ciclo de vida del desarrollo, fortalecer la calidad de los productos y responder con mayor rapidez a las necesidades de las organizaciones y de los usuarios.

Entre los principales beneficios de la implementación de DevOps se encuentran:

- **Reducción de los tiempos de entrega:** la automatización de procesos permite desarrollar, probar y desplegar aplicaciones de manera más ágil.
- **Mejora de la calidad del software:** la integración continua y las pruebas automatizadas contribuyen a detectar errores de forma temprana.
- **Mayor colaboración entre los equipos:** el trabajo conjunto fortalece la comunicación y la responsabilidad compartida durante todo el ciclo de vida del software.
- **Optimización de los procesos:** la automatización disminuye tareas repetitivas, reduce errores y mejora la eficiencia operativa.
- **Mayor capacidad de adaptación al cambio:** facilita la incorporación de nuevas funcionalidades y la respuesta oportuna a los requerimientos del negocio.
- **Fortalecimiento de la continuidad del servicio:** las prácticas de monitoreo y mejora continua contribuyen a mantener aplicaciones más estables y confiables.

La implementación de DevOps no depende únicamente del uso de herramientas tecnológicas, sino también de la adopción de una cultura organizacional basada en la colaboración, la automatización y la mejora continua. La integración de estos elementos permite desarrollar soluciones de software con mayor calidad, optimizar los procesos de trabajo y generar valor para las organizaciones.

Tabla 1. Beneficios de la implementación de DevOps

Beneficio	Aporte al desarrollo de software
Reducción de los tiempos de entrega	Agiliza la construcción, prueba y despliegue de aplicaciones.
Mejora de la calidad	Favorece la detección temprana de errores mediante pruebas automatizadas.
Trabajo colaborativo	Fortalece la comunicación y la responsabilidad compartida entre los equipos.
Automatización de procesos	Reduce tareas manuales y mejora la eficiencia operativa.
Adaptación al cambio	Facilita la incorporación de nuevas funcionalidades y actualizaciones.
Continuidad del servicio	Contribuye a la estabilidad y disponibilidad de las aplicaciones.

La información presentada permite identificar que la implementación de DevOps genera beneficios tanto para los equipos de trabajo como para las organizaciones, al favorecer procesos de desarrollo más ágiles, colaborativos y orientados a la mejora continua.

2. Procesos de desarrollo de software

El desarrollo de software comprende un conjunto de actividades organizadas que permiten transformar una necesidad o un problema en una aplicación o solución informática. Para lograrlo, es necesario establecer procesos que orienten la planificación, el análisis, el diseño, la construcción, las pruebas, la implementación y el mantenimiento del software, garantizando que cada etapa se ejecute de manera ordenada y contribuya al cumplimiento de los objetivos del proyecto.

Los procesos de desarrollo proporcionan una estructura de trabajo que facilita la coordinación entre los integrantes del equipo, la gestión de los recursos y el seguimiento de las actividades. Además, favorecen la definición de responsabilidades, la elaboración de productos intermedios y la aplicación de mecanismos de control que contribuyen a mejorar la calidad del software.

En el contexto de DevOps, estos procesos adquieren un enfoque colaborativo e integrado, donde las actividades de desarrollo y operación dejan de ejecutarse de forma independiente para convertirse en un flujo continuo apoyado por la automatización y la comunicación permanente entre los equipos. Este enfoque favorece la entrega frecuente de nuevas versiones, la detección temprana de errores y la mejora continua durante todo el ciclo de vida del software.

Los procesos de desarrollo de software constituyen la base para implementar prácticas de integración continua, entrega continua y despliegue continuo, las cuales permiten optimizar el trabajo colaborativo y responder con mayor rapidez a las necesidades de las organizaciones y de los usuarios.

2.1. Ciclo de vida del desarrollo de software

El ciclo de vida del desarrollo de software (Software Development Life Cycle - SDLC) es un modelo que organiza las actividades necesarias para planificar, diseñar, desarrollar, probar, implementar y mantener una aplicación informática. Su propósito es proporcionar una estructura que facilite la gestión del proyecto, optimice el uso de los recursos y garantice la calidad del producto final.

La implementación de un ciclo de vida permite definir las actividades que deben ejecutarse en cada etapa, establecer responsabilidades y realizar el seguimiento del avance del proyecto. De esta manera, es posible reducir riesgos, controlar los cambios y asegurar que el software responda a los requisitos definidos por los usuarios y las organizaciones.

Aunque existen diferentes metodologías de desarrollo, la mayoría contempla un conjunto de etapas comunes que orientan la construcción de soluciones de software. Estas etapas son:

- **Planificación:** definición de los objetivos, el alcance, los recursos y el cronograma del proyecto.
- **Análisis de requisitos:** identificación y documentación de las necesidades funcionales y no funcionales que debe cumplir el software.
- **Diseño:** definición de la arquitectura, los componentes, la estructura de datos y la interfaz de la aplicación.
- **Desarrollo:** construcción del software mediante la implementación del código fuente.

- **Pruebas:** verificación del funcionamiento de la aplicación para identificar y corregir errores antes de su implementación.
- **Implementación:** puesta en funcionamiento del software en el entorno de producción.
- **Mantenimiento:** realización de correcciones, actualizaciones y mejoras posteriores a la implementación.

La aplicación ordenada de estas etapas contribuye al desarrollo de software con mayor calidad, facilita el trabajo colaborativo entre los equipos y proporciona una base para la implementación de prácticas DevOps orientadas a la integración y entrega continua.

2.2. Actividades del proceso de desarrollo

El proceso de desarrollo de software comprende un conjunto de actividades que permiten transformar los requisitos definidos en una solución informática funcional. Cada actividad responde a un propósito específico y se desarrolla de manera coordinada con las demás, favoreciendo la organización del proyecto, el control de las tareas y el aseguramiento de la calidad del software.

Aunque las metodologías de desarrollo pueden variar, la mayoría contempla actividades comunes que orientan la construcción, implementación y mantenimiento de las aplicaciones. En la siguiente tabla se presentan las principales actividades que conforman este proceso y el propósito de cada una.

Tabla 2. Actividades del proceso de desarrollo de software

Actividad	Descripción	Propósito
Planificación	Define los objetivos, el alcance, los recursos y el cronograma del proyecto.	Organizar el desarrollo y establecer las condiciones necesarias para ejecutar el proyecto.
Análisis de requisitos	Identifica y documenta las necesidades funcionales y no funcionales del software.	Comprender las necesidades del usuario y establecer las características que debe cumplir la aplicación.
Diseño	Define la arquitectura, los componentes, las bases de datos y las interfaces del sistema.	Establecer la estructura técnica que servirá de base para el desarrollo del software.
Desarrollo	Implementa el código fuente de acuerdo con los requisitos y el diseño establecidos.	Construir la aplicación y desarrollar sus funcionalidades.
Pruebas	Verifica el funcionamiento del software e identifica posibles errores.	Validar que la aplicación cumpla los requisitos definidos y funcione correctamente.
Implementación	Instala y configura la aplicación en el entorno de producción.	Poner el software a disposición de los usuarios para su utilización.
Mantenimiento	Realiza correcciones, actualizaciones y mejoras posteriores a la implementación.	Garantizar el funcionamiento continuo y la evolución del software según las necesidades de la organización.

La información presentada permite identificar que las actividades del proceso de desarrollo se encuentran estrechamente relacionadas y se ejecutan de forma coordinada para obtener aplicaciones funcionales y de calidad. En el enfoque DevOps, estas actividades se integran mediante prácticas colaborativas y procesos automatizados que favorecen la entrega continua y la mejora permanente del software.

2.3. Roles y responsabilidades del equipo de desarrollo

El desarrollo de software requiere la participación de profesionales con diferentes conocimientos y responsabilidades, quienes trabajan de manera coordinada para planificar, construir, implementar y mantener las aplicaciones. Cada integrante del equipo aporta habilidades específicas que contribuyen al cumplimiento de los objetivos del proyecto y a la entrega de productos de calidad.

En el enfoque DevOps, los roles mantienen sus funciones principales, pero fortalecen el trabajo colaborativo y la comunicación permanente entre las áreas de desarrollo, operaciones, pruebas y seguridad. Este modelo promueve la responsabilidad compartida y la integración de actividades durante todo el ciclo de vida del software.

A continuación, se describen algunos de los principales roles que participan en el desarrollo de software y sus responsabilidades dentro de un entorno de trabajo colaborativo.

Tabla 3. Roles y responsabilidades del equipo de desarrollo de software

Rol	Responsabilidades principales	Participación en DevOps
Analista de requisitos	Identificar, documentar y validar las necesidades del cliente o usuario.	Facilita la comunicación entre las partes interesadas y el equipo de desarrollo.
Arquitecto de software	Definir la arquitectura, los componentes y las tecnologías de la solución.	Diseña soluciones que facilitan la integración, el despliegue y la escalabilidad.
Desarrollador	Implementar el código fuente y corregir errores de la aplicación.	Integra cambios frecuentes al repositorio y participa en procesos de integración continua.
Ingeniero de pruebas (QA)	Diseñar y ejecutar pruebas para verificar la calidad del software.	Automatiza pruebas y valida el correcto funcionamiento de las aplicaciones antes de su despliegue.
Administrador de operaciones (Ops)	Gestionar la infraestructura, la implementación y el funcionamiento de las aplicaciones.	Colabora con el equipo de desarrollo para garantizar la disponibilidad y continuidad de los servicios.
Administrador de proyectos o Scrum Master	Coordinar las actividades del equipo y realizar el seguimiento del proyecto.	Favorece la colaboración, la comunicación y la mejora continua de los procesos.

El desarrollo de software requiere la participación coordinada de diferentes profesionales. En DevOps, estos roles fortalecen el trabajo colaborativo, comparten responsabilidades y contribuyen conjuntamente al desarrollo, implementación y operación de aplicaciones de software.

2.4. Artefactos de software

Durante el desarrollo de software se generan diferentes productos o documentos que registran la información necesaria para planificar, construir, verificar y mantener una aplicación. Estos productos se conocen como artefactos de software y constituyen evidencias del trabajo realizado en las distintas etapas del ciclo de vida del desarrollo.

Los artefactos facilitan la comunicación entre los integrantes del equipo, permiten documentar las decisiones tomadas durante el proyecto y proporcionan información para el seguimiento, la validación y el mantenimiento de la solución desarrollada. Asimismo, favorecen la trazabilidad de los requisitos y contribuyen al aseguramiento de la calidad del software.

Entre los artefactos más utilizados durante el desarrollo de software se encuentran:

- **Documento de requisitos:** reúne las necesidades funcionales y no funcionales que debe satisfacer la aplicación. Sirve como referencia para orientar el diseño, el desarrollo y la validación del software, además de facilitar la comunicación entre el cliente y el equipo de trabajo.

- **Modelo de diseño:** describe la arquitectura del sistema, los componentes, las interfaces, las estructuras de datos y la interacción entre los diferentes elementos de la aplicación. Constituye la base técnica para el desarrollo del software.
- **Código fuente:** corresponde al conjunto de instrucciones escritas en un lenguaje de programación que implementan las funcionalidades de la aplicación. Es el principal artefacto del desarrollo y evoluciona continuamente durante el proyecto.
- **Casos de prueba:** documentan los escenarios, datos y resultados esperados que permiten verificar el correcto funcionamiento de la aplicación. Su ejecución facilita la identificación de errores y la validación de los requisitos definidos.
- **Manual técnico:** contiene información sobre la instalación, configuración, arquitectura, mantenimiento y administración del sistema. Está dirigido al personal técnico responsable del soporte y la operación de la aplicación.
- **Manual de usuario:** proporciona instrucciones para utilizar la aplicación, describiendo sus funcionalidades, procedimientos y recomendaciones de uso. Su propósito es facilitar la interacción de los usuarios con el sistema.

Cada uno de estos artefactos cumple una función específica y contribuye a mantener la organización, la calidad y la trazabilidad durante el desarrollo del software.

Ejemplo de utilización de artefactos de software

Durante el desarrollo de una aplicación para la gestión de citas médicas, el equipo de trabajo utiliza diferentes artefactos en cada etapa del proyecto. Inicialmente,

elabora un documento de requisitos para registrar las necesidades del sistema, como el agendamiento de citas y la consulta de historiales médicos. Posteriormente, desarrolla un modelo de diseño que define la arquitectura de la aplicación, las interfaces y la estructura de la base de datos.

Una vez aprobado el diseño, los desarrolladores construyen el código fuente de la aplicación y el equipo de calidad elabora los casos de prueba para verificar que las funcionalidades operen correctamente. Finalmente, antes de la entrega del sistema, se preparan el manual técnico, dirigido al personal encargado de la administración y el mantenimiento, y el manual de usuario, que orienta a los usuarios finales en la utilización de la aplicación.

Este ejemplo permite identificar que los artefactos de software se generan de manera progresiva durante el desarrollo de un proyecto y que cada uno cumple una función específica para documentar el proceso, facilitar el trabajo colaborativo y garantizar la calidad de la solución desarrollada.

2.5. Herramientas para el desarrollo de software

El desarrollo de software requiere el apoyo de herramientas que facilitan la planificación, la construcción, las pruebas, la implementación y el mantenimiento de las aplicaciones. Estas herramientas permiten optimizar las actividades del equipo de trabajo, mejorar la productividad y favorecer la colaboración entre los diferentes participantes del proyecto.

La selección de las herramientas depende de las características del proyecto, la metodología de desarrollo y las tecnologías utilizadas. En la actualidad, existen

soluciones especializadas para la gestión de requisitos, el desarrollo del código, el control de versiones, la automatización de pruebas y la administración de los procesos de integración y despliegue continuo.

Tabla 4. Herramientas utilizadas en el desarrollo de software

Categoría	Ejemplos de herramientas	Aplicación principal
Gestión de proyectos	Jira, Trello y Azure Boards.	Planificar actividades, gestionar tareas y realizar el seguimiento del proyecto.
Desarrollo de código	Visual Studio Code, Eclipse e IntelliJ IDEA	Crear, editar y depurar el código fuente de las aplicaciones.
Control de versiones	Git.	Gestionar los cambios realizados sobre el código fuente y facilitar el trabajo colaborativo.
Repositorios remotos	GitHub y GitLab.	Almacenar el código fuente y apoyar la colaboración entre los integrantes del equipo.
Pruebas	JUnit, Selenium y Postman.	Verificar el funcionamiento de las aplicaciones mediante pruebas funcionales y automatizadas.
Integración y automatización	Jenkins.	Automatizar procesos de construcción, pruebas e integración continua.

Categoría	Ejemplos de herramientas	Aplicación principal
Contenedores	Docker.	Crear entornos de ejecución consistentes para el desarrollo y despliegue de aplicaciones.

Cada herramienta cumple una función específica dentro del proceso de desarrollo de software. Su integración favorece el trabajo colaborativo, optimiza la gestión del código fuente y contribuye a la automatización de actividades relacionadas con la construcción, las pruebas y el despliegue de las aplicaciones.

3. Contenedores en entornos de desarrollo

El desarrollo de aplicaciones modernas requiere entornos de trabajo que faciliten la creación, ejecución y mantenimiento del software de manera consistente en diferentes plataformas. En este contexto, los contenedores se han convertido en una tecnología ampliamente utilizada, ya que permiten empaquetar una aplicación junto con sus dependencias, bibliotecas y configuraciones necesarias para su funcionamiento.

A diferencia de otros mecanismos de virtualización, los contenedores comparten el sistema operativo del equipo anfitrión, lo que reduce el consumo de recursos y permite ejecutar múltiples aplicaciones de forma aislada y eficiente. Esta característica facilita la portabilidad de las aplicaciones entre diferentes entornos de desarrollo, pruebas y producción, garantizando un comportamiento consistente en cada uno de ellos.

En el enfoque DevOps, los contenedores favorecen la automatización de los procesos de desarrollo y despliegue, simplifican la administración de las aplicaciones y contribuyen a la integración continua y la entrega continua del software. Su utilización también facilita la colaboración entre los equipos de desarrollo y operaciones, al proporcionar entornos consistentes para la construcción, validación e implementación de las soluciones.

3.1. Características de los contenedores

Los contenedores presentan características que los convierten en una tecnología ampliamente utilizada durante el desarrollo y la implementación de aplicaciones. Estas

características permiten crear entornos consistentes, optimizar el uso de los recursos y facilitar la ejecución del software en diferentes plataformas.

Entre las principales características de los contenedores se encuentran:

- **Aislamiento:** cada contenedor ejecuta una aplicación de manera independiente, evitando conflictos con otras aplicaciones o servicios que se encuentren en el mismo sistema.
- **Portabilidad:** pueden trasladarse entre diferentes entornos de desarrollo, pruebas y producción, manteniendo el mismo comportamiento y reduciendo los problemas asociados a la configuración del entorno.
- **Ligereza:** utilizan menos recursos que otros mecanismos de virtualización, ya que comparten el sistema operativo del equipo anfitrión en lugar de ejecutar un sistema operativo completo para cada instancia.
- **Escalabilidad:** permiten crear o eliminar instancias de una aplicación de forma rápida, facilitando la atención de cambios en la demanda de los usuarios.
- **Consistencia:** garantizan que la aplicación se ejecute con las mismas dependencias, bibliotecas y configuraciones en cualquier entorno donde se implemente.
- **Rapidez de implementación:** su creación y puesta en funcionamiento requieren menos tiempo, lo que favorece la automatización de los procesos de desarrollo y despliegue.

Estas características hacen que los contenedores sean una alternativa eficiente para el desarrollo de aplicaciones modernas, especialmente en entornos donde se

requiere automatizar procesos, optimizar recursos y garantizar la estabilidad de las aplicaciones durante todo su ciclo de vida.

3.2. Beneficios del uso de contenedores

La utilización de contenedores aporta ventajas significativas durante el desarrollo, las pruebas y la implementación de aplicaciones. Su capacidad para ejecutar el software de manera consistente en diferentes entornos facilita el trabajo colaborativo, optimiza el uso de los recursos tecnológicos y fortalece los procesos de integración y despliegue continuo.

Los beneficios de los contenedores no solo favorecen a los equipos de desarrollo, sino también a las organizaciones, al proporcionar mayor flexibilidad para administrar las aplicaciones y responder con rapidez a las necesidades del negocio.

Tabla 5. Beneficios del uso de contenedores

Beneficio	Descripción	Impacto en el desarrollo de software
Portabilidad	Permite ejecutar una aplicación en diferentes entornos sin modificar su configuración.	Reduce problemas de compatibilidad entre desarrollo, pruebas y producción.
Optimización de recursos	Comparte el sistema operativo del equipo anfitrión, disminuyendo el consumo de memoria y procesamiento.	Permite ejecutar más aplicaciones utilizando la misma infraestructura.

Beneficio	Descripción	Impacto en el desarrollo de software
Escalabilidad	Facilita la creación o eliminación de instancias de una aplicación según la demanda.	Favorece la disponibilidad y el rendimiento de los servicios.
Rapidez de implementación	Reduce el tiempo necesario para crear y poner en funcionamiento nuevos entornos.	Agiliza los procesos de desarrollo, pruebas y despliegue.
Consistencia entre entornos	Mantiene las mismas dependencias y configuraciones durante todo el ciclo de vida de la aplicación.	Disminuye errores asociados a diferencias entre entornos de trabajo.
Automatización	Se integra con herramientas y procesos de integración y despliegue continuo.	Favorece la implementación de prácticas DevOps y la entrega continua de software.

La información presentada permite identificar que los contenedores contribuyen a optimizar el desarrollo de software mediante la creación de entornos consistentes, el aprovechamiento eficiente de los recursos y la automatización de procesos. Estas ventajas favorecen la calidad de las aplicaciones y fortalecen la capacidad de respuesta de las organizaciones frente a los cambios tecnológicos.

3.3. Aplicaciones de los contenedores en DevOps

En el enfoque DevOps, los contenedores se utilizan para apoyar las diferentes actividades del desarrollo, las pruebas, la integración y el despliegue de aplicaciones. Su utilización facilita la creación de entornos consistentes, favorece la automatización de procesos y contribuye a mejorar la calidad y disponibilidad del software.

Entre las principales aplicaciones de los contenedores en DevOps se encuentran:

- **Desarrollo de aplicaciones:** permiten que todos los integrantes del equipo trabajen en entornos con las mismas configuraciones, dependencias y versiones de software, reduciendo problemas de compatibilidad.
- **Integración continua (CI):** facilitan la construcción y validación automática de nuevas versiones de la aplicación cada vez que se realizan cambios en el código fuente.
- **Ejecución de pruebas:** posibilitan la creación de entornos temporales donde las pruebas funcionales, de integración o de rendimiento se ejecutan de forma aislada, garantizando resultados confiables.
- **Entrega y despliegue continuo (CD):** permiten distribuir aplicaciones entre los diferentes entornos de desarrollo, pruebas y producción de manera rápida, consistente y automatizada.
- **Escalabilidad de aplicaciones:** facilitan la creación de nuevas instancias de una aplicación cuando aumenta la demanda de los usuarios, contribuyendo a mantener la disponibilidad del servicio.
- **Implementación de herramientas de apoyo:** posibilitan ejecutar herramientas utilizadas para el análisis de calidad del código, las pruebas

automatizadas y la seguridad, sin necesidad de realizar instalaciones independientes en cada servidor.

Ejemplo de aplicación de contenedores en DevOps

Durante el desarrollo de una aplicación web, cada integrante del equipo utiliza un contenedor con la misma configuración del proyecto. Cuando un desarrollador incorpora cambios al código fuente, el sistema inicia un proceso automatizado que construye una nueva versión de la aplicación, ejecuta las pruebas correspondientes y prepara el despliegue en el entorno definido. Este procedimiento reduce errores asociados a diferencias entre entornos y favorece la entrega continua de nuevas versiones del software.

La utilización de contenedores en DevOps fortalece la automatización de los procesos, mejora la colaboración entre los equipos y facilita la entrega de aplicaciones más estables, consistentes y adaptables a las necesidades de las organizaciones.

3.4. Uso de contenedores en entornos DevOps

En los entornos DevOps, la creación y administración de contenedores suele realizarse mediante herramientas especializadas. Entre ellas, Docker es una de las más utilizadas, ya que permite construir imágenes, ejecutar contenedores y automatizar procesos relacionados con el desarrollo, las pruebas y el despliegue de aplicaciones.

Para crear una imagen de un contenedor se utiliza un archivo denominado Dockerfile, el cual contiene las instrucciones necesarias para preparar el entorno donde se ejecutará la aplicación.

A continuación, se presenta un ejemplo simplificado de un Dockerfile:

```
FROM node:18-alpine
```

```
WORKDIR /app
```

```
COPY . .
```

```
RUN npm install
```

```
EXPOSE 3000
```

```
CMD ["node", "server.js"]
```

En este ejemplo:

- **FROM** especifica la imagen base que utilizará el contenedor.
- **WORKDIR** establece el directorio de trabajo.
- **COPY** copia los archivos de la aplicación al contenedor.
- **RUN** ejecuta comandos durante la construcción de la imagen.
- **EXPOSE** indica el puerto utilizado por la aplicación.
- **CMD** define el comando que se ejecutará al iniciar el contenedor.

Los comandos de Docker permiten gestionar el ciclo de vida de los contenedores durante el desarrollo y la implementación de aplicaciones. A continuación, se presentan algunos de los comandos básicos utilizados en entornos DevOps.

Tabla 6. Comandos básicos de Docker

Comando	Propósito	Ejemplo
docker build	Construir una imagen a partir de un Dockerfile.	docker build -t mi-aplicacion .
docker run	Crear y ejecutar un contenedor desde una imagen.	docker run -d -p 8080:3000 mi-aplicacion
docker ps	Mostrar los contenedores en ejecución.	docker ps
docker stop	Detener un contenedor en ejecución.	docker stop mi-contenedor
docker images	Listar las imágenes almacenadas localmente.	docker images
docker logs	Consultar los registros de un contenedor.	docker logs mi-contenedor

La utilización de herramientas como Docker facilita la administración de contenedores durante el desarrollo y la operación de aplicaciones. Su integración con las prácticas DevOps favorece la automatización de procesos, la consistencia entre entornos y la entrega continua de software.

4. Integración, entrega y despliegue continuo (CI/CD)

El desarrollo moderno de aplicaciones requiere procesos que permitan incorporar cambios al software de manera ágil, segura y con altos niveles de calidad. En este contexto, las prácticas de integración continua, entrega continua y despliegue continuo, conocidas de forma conjunta como CI/CD, constituyen un conjunto de estrategias que automatizan diferentes actividades del ciclo de vida del desarrollo, facilitando la construcción, validación y distribución de las aplicaciones.

La adopción de CI/CD permite que los cambios realizados al código fuente sean integrados, verificados y preparados para su implementación de manera frecuente, reduciendo los tiempos de entrega y disminuyendo los errores asociados a los procesos manuales. Estas prácticas fortalecen la colaboración entre los equipos de desarrollo y operaciones, favoreciendo la entrega continua de software y la mejora permanente de los procesos.

En los entornos DevOps, la integración, la entrega y el despliegue continuo se apoyan en herramientas que automatizan actividades como la compilación del código, la ejecución de pruebas, la generación de artefactos y la implementación de aplicaciones en diferentes entornos. Esta automatización contribuye a incrementar la eficiencia del desarrollo, mejorar la calidad del software y responder con mayor rapidez a las necesidades de las organizaciones y de los usuarios.

4.1. Integración continua (CI)

La integración continua (Continuous Integration - CI) es una práctica de desarrollo de software que consiste en integrar de manera frecuente los cambios realizados al

código fuente dentro de un repositorio compartido. Cada vez que un desarrollador incorpora nuevas funcionalidades o realiza modificaciones, estos cambios son verificados automáticamente mediante procesos de compilación y pruebas, con el propósito de identificar errores de forma temprana.

La integración continua favorece el trabajo colaborativo entre los integrantes del equipo de desarrollo, ya que permite detectar incompatibilidades antes de que afecten el funcionamiento de la aplicación. De esta manera, se reducen los conflictos durante la integración del código y se incrementa la estabilidad del proyecto.

Generalmente, el proceso de integración continua comprende las siguientes actividades:

- **Incorporación de cambios al repositorio:** los desarrolladores integran periódicamente las modificaciones realizadas al código fuente.
- **Compilación automática:** el sistema construye una nueva versión de la aplicación utilizando el código actualizado.
- **Ejecución de pruebas automatizadas:** se verifica el correcto funcionamiento del software mediante pruebas previamente definidas.
- **Generación de resultados:** el sistema informa si la integración fue exitosa o si existen errores que deben corregirse antes de continuar con el proceso.

La implementación de la integración continua permite detectar problemas en etapas tempranas del desarrollo, mejorar la calidad del software y preparar la aplicación para los procesos de entrega y despliegue continuo.

Ejemplo de integración continua

Un desarrollador incorpora una nueva funcionalidad a una aplicación y registra los cambios en el repositorio del proyecto. De forma automática, el sistema inicia la compilación del código y ejecuta las pruebas definidas. Si todas las verificaciones son satisfactorias, la nueva versión queda disponible para continuar con las siguientes etapas del proceso de desarrollo.

Este ejemplo permite identificar que la integración continua automatiza la validación de los cambios realizados al software, favoreciendo la detección temprana de errores y fortaleciendo la calidad del producto desarrollado.

4.2. Entrega continua (CD)

La entrega continua (Continuous Delivery - CD) es una práctica que amplía el proceso de integración continua, preparando automáticamente las aplicaciones para su implementación en diferentes entornos.

Después de integrar y verificar los cambios en el código fuente, el software queda listo para ser entregado de manera rápida, confiable y controlada cuando la organización lo requiera.

En la entrega continua, los procesos de construcción, validación y preparación de la aplicación se realizan de forma automatizada. Sin embargo, la implementación en el entorno de producción suele requerir una aprobación o decisión por parte del equipo responsable antes de publicar la nueva versión para los usuarios.

Generalmente, la entrega continua comprende las siguientes actividades:

- **Preparación de la aplicación:** genera una versión estable del software después de superar las pruebas de integración.
- **Ejecución de pruebas adicionales:** verifica aspectos funcionales, de rendimiento o seguridad antes de la implementación.
- **Generación de artefactos de despliegue:** crea los paquetes o imágenes necesarios para distribuir la aplicación.
- **Disponibilidad para la implementación:** deja la aplicación preparada para ser desplegada cuando se autorice su publicación.

La entrega continua permite reducir los tiempos necesarios para publicar nuevas versiones del software, mejorar la calidad de las implementaciones y disminuir los riesgos asociados a los procesos manuales.

Ejemplo de entrega continua

Después de validar una nueva versión de una aplicación, el sistema genera automáticamente el paquete de despliegue y lo almacena en un repositorio. La versión queda preparada para ser implementada en el entorno de producción una vez el responsable del proyecto autorice su publicación.

Este ejemplo permite identificar que la entrega continua automatiza la preparación del software para su implementación, garantizando que cada versión validada se encuentre disponible para ser desplegada de manera rápida y controlada.

4.3. Despliegue continuo

El despliegue continuo (Continuous Deployment) es una práctica mediante la cual las nuevas versiones de una aplicación se implementan automáticamente en el entorno de producción una vez han superado todas las pruebas y validaciones establecidas. A diferencia de la entrega continua, este proceso no requiere una aprobación manual para publicar los cambios, ya que la implementación se realiza de forma automatizada.

El despliegue continuo permite que las mejoras, correcciones y nuevas funcionalidades lleguen a los usuarios con mayor rapidez, manteniendo un flujo permanente de actualizaciones. Para lograrlo, es necesario contar con procesos de automatización confiables y mecanismos de validación que garanticen la estabilidad y el correcto funcionamiento de la aplicación antes de su publicación.

Generalmente, el despliegue continuo comprende las siguientes actividades:

- **Validación automática del software:** verifica que la aplicación haya superado satisfactoriamente las pruebas definidas.
- **Implementación automática:** publica la nueva versión en el entorno de producción sin intervención manual.
- **Verificación del funcionamiento:** comprueba que la aplicación opere correctamente después del despliegue.
- **Monitoreo continuo:** supervisa el comportamiento de la aplicación para identificar posibles incidentes y facilitar su atención oportuna.

La implementación del despliegue continuo permite reducir los tiempos de publicación, agilizar la entrega de nuevas funcionalidades y fortalecer la capacidad de respuesta frente a las necesidades de los usuarios y de las organizaciones.

Ejemplo de despliegue continuo

Una aplicación supera automáticamente las pruebas de integración, seguridad y funcionamiento. Una vez finalizadas estas validaciones, el sistema publica la nueva versión en el entorno de producción sin intervención del equipo de desarrollo. Posteriormente, el comportamiento de la aplicación es monitoreado para verificar que el servicio continúe funcionando correctamente.

Este ejemplo permite identificar que el despliegue continuo automatiza la publicación de nuevas versiones del software, favoreciendo actualizaciones frecuentes y reduciendo el tiempo requerido para poner las aplicaciones a disposición de los usuarios. A continuación, se resumen las principales diferencias entre la integración continua, la entrega continua y el despliegue continuo, destacando el propósito de cada práctica dentro del enfoque DevOps.

Tabla 7. Diferencias entre integración continua, entrega continua y despliegue continuo

Práctica	Objetivo principal	Implementación en producción
Integración continua (CI)	Integrar y validar los cambios en el código fuente.	No aplica.
Entrega continua (CD)	Preparar automáticamente la aplicación para su implementación.	Requiere aprobación manual para publicar la versión.
Despliegue continuo	Publicar automáticamente la aplicación una vez superadas las validaciones.	Se realiza de forma automática, sin intervención manual.

4.4. Beneficios de la automatización de procesos CI/CD

La automatización de los procesos de integración, entrega y despliegue continuo constituye uno de los principales fundamentos del enfoque DevOps. Mediante la ejecución automática de actividades como la compilación, las pruebas, la generación de artefactos y la implementación de aplicaciones, es posible optimizar el desarrollo de software, reducir la intervención manual y mejorar la confiabilidad de los procesos.

La implementación de prácticas CI/CD permite responder con mayor rapidez a los cambios en los proyectos, fortalecer la colaboración entre los equipos de desarrollo y operaciones y garantizar que las nuevas versiones del software sean entregadas con mayores niveles de calidad y estabilidad.

Tabla 8. Beneficios de la automatización de procesos CI/CD

Beneficio	Descripción	Impacto en el desarrollo de software
Reducción de errores	Disminuye las fallas ocasionadas por actividades manuales repetitivas.	Incrementa la calidad y confiabilidad del software.
Mayor rapidez en las entregas	Automatiza procesos de construcción, pruebas e implementación.	Reduce el tiempo necesario para publicar nuevas versiones.
Mejora de la calidad	Permite ejecutar pruebas de manera frecuente y consistente.	Facilita la detección temprana de errores y defectos.

Beneficio	Descripción	Impacto en el desarrollo de software
Trabajo colaborativo	Integra las actividades de desarrollo y operaciones mediante procesos compartidos.	Favorece la comunicación y la coordinación entre los equipos.
Eficiencia operativa	Optimiza el uso de recursos al disminuir tareas manuales y repetitivas.	Incrementa la productividad durante el ciclo de desarrollo.
Mayor confiabilidad en los despliegues	Automatiza la implementación de aplicaciones utilizando procesos previamente validados.	Reduce riesgos durante la publicación de nuevas versiones.

La automatización de procesos CI/CD constituye un elemento clave para el desarrollo moderno de aplicaciones, ya que facilita la integración de cambios, fortalece la calidad del software y mejora la capacidad de respuesta frente a las necesidades de las organizaciones y de los usuarios.

4.5. Flujo de trabajo en entornos DevOps

El flujo de trabajo en DevOps integra las actividades de desarrollo, pruebas, implementación y operación mediante procesos automatizados que favorecen la entrega continua de software. Este enfoque permite que los cambios realizados al código fuente avancen de forma ordenada a través de las diferentes etapas del ciclo de desarrollo, garantizando la calidad de las aplicaciones antes de su publicación.

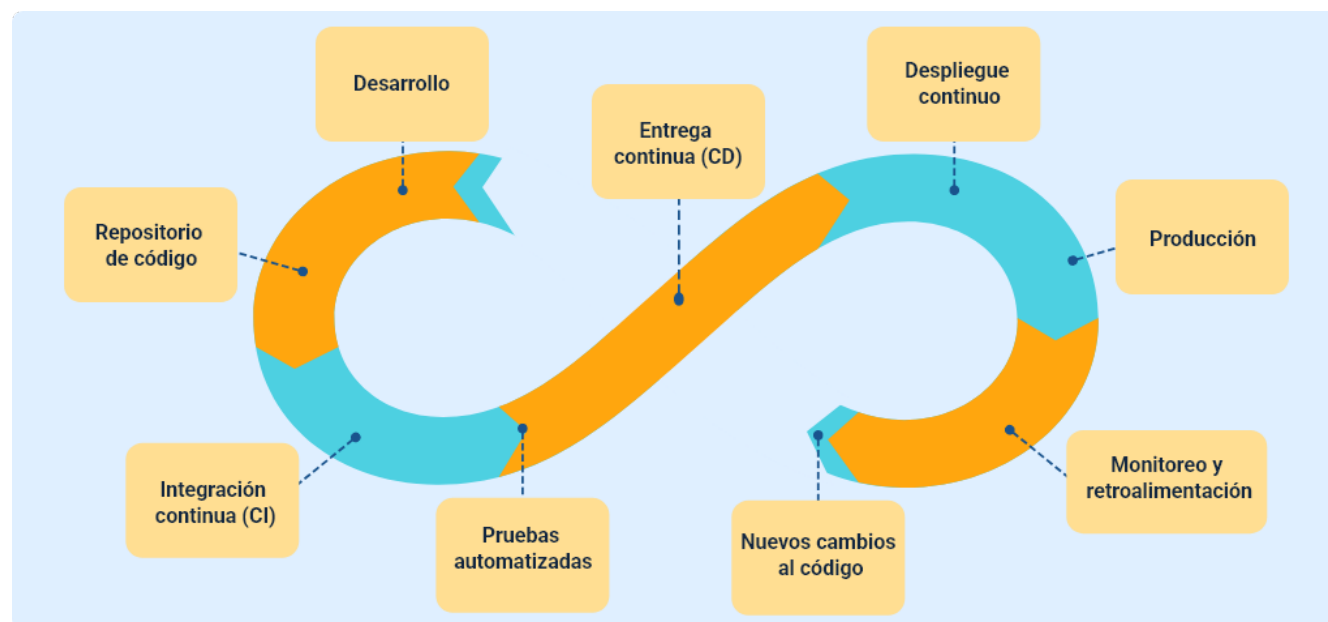
En un entorno DevOps, cada modificación incorporada al proyecto inicia una secuencia de actividades que incluye la integración del código, la ejecución de pruebas automatizadas, la preparación de nuevas versiones y su implementación en los entornos correspondientes. Una vez la aplicación se encuentra en producción, su funcionamiento es monitoreado continuamente para identificar oportunidades de mejora y atender oportunamente posibles incidentes.

De manera general, el flujo de trabajo comprende las siguientes etapas:

- a. **Desarrollo:** implementación de nuevas funcionalidades o corrección de errores en el código fuente.
- b. **Integración continua:** incorporación y validación automática de los cambios realizados.
- c. **Pruebas automatizadas:** verificación del funcionamiento de la aplicación mediante diferentes tipos de pruebas.
- d. **Entrega continua:** preparación de la aplicación para su implementación en los diferentes entornos.
- e. **Despliegue continuo:** publicación automática de nuevas versiones cuando se cumplen los criterios establecidos.
- f. **Monitoreo:** supervisión permanente del comportamiento de la aplicación para garantizar su disponibilidad, rendimiento y estabilidad.

A continuación, se presenta un esquema general del flujo de trabajo utilizado en un entorno DevOps.

Figura 4. Flujo de trabajo en un entorno DevOps



Este proceso permite integrar el desarrollo, las pruebas, la implementación y el monitoreo de las aplicaciones dentro de un ciclo continuo orientado a la mejora permanente del software.

5. Gestión de versiones con Git

El desarrollo de software implica la realización constante de cambios sobre el código fuente, ya sea para incorporar nuevas funcionalidades, corregir errores o mejorar el rendimiento de las aplicaciones. Para administrar estas modificaciones de manera organizada y facilitar el trabajo colaborativo, se utilizan sistemas de control de versiones que permiten registrar la evolución del proyecto y mantener un historial de los cambios realizados.

Entre las herramientas de control de versiones más utilizadas se encuentra Git, un sistema distribuido que permite gestionar el código fuente, coordinar el trabajo de varios desarrolladores y recuperar versiones anteriores cuando es necesario. Su utilización favorece la trazabilidad de los cambios, reduce los conflictos durante el desarrollo y constituye un elemento fundamental dentro de las prácticas de integración continua e implementación de DevOps.

La gestión de versiones con Git también facilita la colaboración entre los integrantes del equipo mediante el uso de repositorios compartidos, estrategias de ramificación y mecanismos para integrar los cambios de forma controlada. Estas capacidades permiten desarrollar aplicaciones de manera más organizada, mantener la estabilidad del código y responder con mayor agilidad a las necesidades de los proyectos de software.

5.1. Fundamentos del control de versiones

El desarrollo de software implica la creación y modificación constante del código fuente durante todo el ciclo de vida de una aplicación. A medida que un

proyecto evoluciona, diferentes desarrolladores pueden realizar cambios de manera simultánea, incorporar nuevas funcionalidades, corregir errores o adaptar el software a nuevas necesidades. En este contexto, resulta necesario contar con mecanismos que permitan registrar, organizar y controlar cada una de estas modificaciones.

El control de versiones es un conjunto de prácticas y herramientas que permite gestionar los cambios realizados sobre los archivos de un proyecto, conservando un historial de las modificaciones efectuadas a lo largo del tiempo. Gracias a este proceso, es posible identificar quién realizó un cambio, cuándo se efectuó y, si es necesario, recuperar versiones anteriores del software.

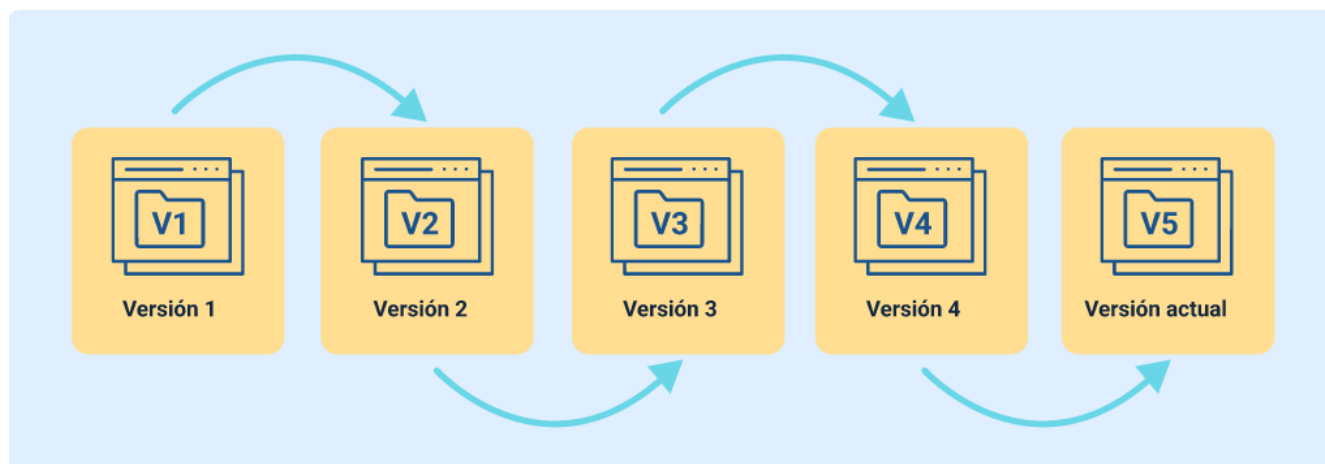
La utilización de sistemas de control de versiones favorece el trabajo colaborativo, facilita la integración de cambios realizados por diferentes desarrolladores y reduce el riesgo de pérdida de información. Además, proporciona trazabilidad sobre la evolución del proyecto y permite mantener un registro organizado del desarrollo de las aplicaciones.

Entre las principales funciones del control de versiones se encuentran:

- Registrar los cambios realizados en el código fuente durante el desarrollo del proyecto.
- Conservar el historial de versiones, facilitando la consulta y recuperación de estados anteriores del software.
- Favorecer el trabajo colaborativo, permitiendo que varios desarrolladores trabajen sobre un mismo proyecto de forma organizada.
- Controlar los cambios, identificando las modificaciones realizadas en cada versión.

- Facilitar la recuperación del código en caso de errores, pérdidas de información o necesidad de volver a una versión anterior.

Figura 5. Funcionamiento general del control de versiones



El control de versiones registra la evolución de un proyecto de software, conservando el historial de los cambios realizados y permitiendo recuperar versiones anteriores cuando sea necesario.

5.2. Git como soporte para la integración continua

La integración continua requiere que los cambios realizados al código fuente sean incorporados de forma frecuente, organizada y segura dentro de un repositorio compartido. Para lograrlo, se utilizan sistemas de control de versiones como Git, los cuales permiten registrar las modificaciones realizadas por los desarrolladores, mantener un historial de cambios y facilitar el trabajo colaborativo durante el desarrollo de software.

En un entorno de integración continua, Git actúa como el punto de partida del proceso automatizado. Cada vez que un desarrollador incorpora cambios al repositorio, estos pueden activar procesos de compilación, ejecución de pruebas y validación del software mediante herramientas de automatización. De esta manera, es posible identificar errores de forma temprana y garantizar que el código integrado cumpla con los criterios de calidad establecidos.

El uso de Git como soporte para la integración continua ofrece, entre otras, las siguientes ventajas:

- **Registro de cambios:** conserva el historial de las modificaciones realizadas sobre el código fuente.
- **Trabajo colaborativo:** permite que varios desarrolladores participen simultáneamente en un mismo proyecto.
- **Integración de cambios:** facilita la incorporación de nuevas funcionalidades mediante procesos controlados.
- **Trazabilidad:** permite identificar quién realizó cada modificación y en qué momento.
- **Automatización:** posibilita la ejecución automática de procesos de construcción, pruebas e integración cuando se incorporan cambios al repositorio.

Como ejemplo, algunos de los comandos básicos utilizados para registrar cambios en Git son los siguientes:

- **git add:** agrega los archivos modificados al área de preparación (staging).
- **git commit:** registra los cambios realizados en el repositorio local.

- **git push:** envía los cambios registrados al repositorio remoto.

Estos comandos forman parte del flujo de trabajo habitual en Git y constituyen el punto de partida para los procesos de integración continua implementados en los proyectos de desarrollo de software.

Ejemplo de integración entre Git y CI

Un desarrollador realiza modificaciones en el código fuente de una aplicación y registra los cambios mediante un git commit. Posteriormente, utiliza git push para enviar la nueva versión al repositorio remoto. A partir de esta acción, una herramienta de integración continua inicia automáticamente la compilación de la aplicación y la ejecución de las pruebas definidas para verificar que los cambios puedan integrarse correctamente al proyecto.

Este ejemplo permite identificar que Git no solo permite gestionar versiones del código fuente, sino que también actúa como el mecanismo que desencadena los procesos automatizados de integración continua dentro del enfoque DevOps.

Como complemento al proceso de aprendizaje, se recomienda explorar la plataforma Learn Git Branching, la cual ofrece ejercicios interactivos para comprender el funcionamiento de Git y practicar operaciones relacionadas con el control de versiones, la gestión de ramas y la integración de cambios en proyectos de desarrollo de software. https://learngitbranching.js.org/?locale=es_ES

5.3. Repositorios remotos con GitHub y GitLab

El desarrollo de aplicaciones de software suele realizarse de manera colaborativa, involucrando equipos de trabajo que participan en diferentes etapas del proyecto. Para facilitar esta colaboración, resulta necesario disponer de un espacio centralizado donde se almacene el código fuente y se registren los cambios realizados por cada integrante del equipo. En este contexto, los repositorios remotos constituyen una herramienta fundamental para la gestión de proyectos de software.

Un repositorio remoto es un espacio de almacenamiento ubicado en un servidor al que pueden acceder los desarrolladores autorizados mediante una conexión a Internet. A diferencia del repositorio local, que se encuentra en el equipo de cada desarrollador, el repositorio remoto centraliza la información del proyecto y permite compartir, sincronizar y administrar las diferentes versiones del código fuente.

Las plataformas GitHub y GitLab son dos de las soluciones más utilizadas para alojar repositorios remotos basados en Git. Además de almacenar el código fuente, estas plataformas proporcionan herramientas que facilitan el trabajo colaborativo, la administración de proyectos, la revisión de cambios y la integración con procesos de automatización propios del enfoque DevOps.

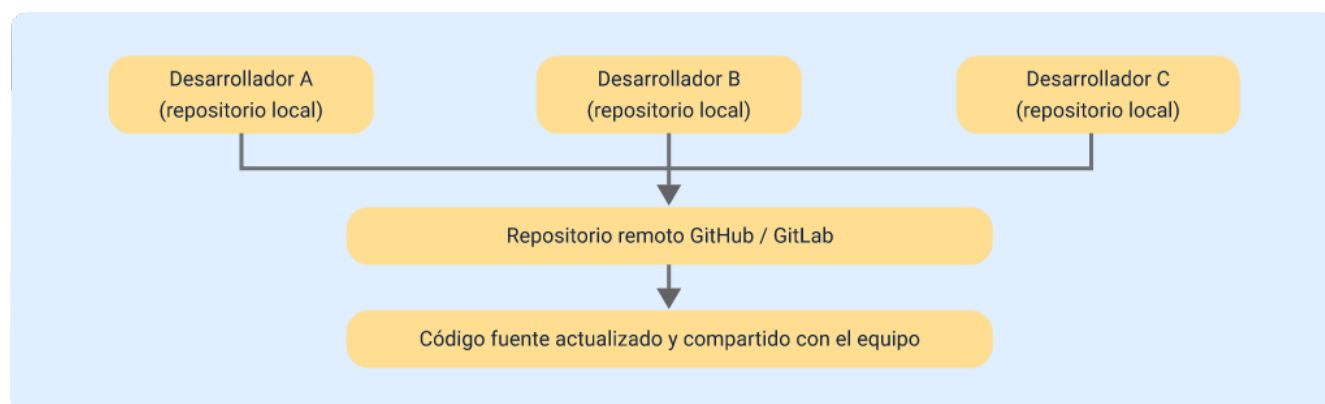
El trabajo con un repositorio remoto sigue un proceso organizado que permite mantener sincronizado el código fuente entre todos los integrantes del equipo de desarrollo. De manera general, este proceso comprende las siguientes etapas:

- **Creación del repositorio:** se crea un repositorio remoto en una plataforma como GitHub o GitLab para almacenar el proyecto.

- **Clonación del repositorio:** cada desarrollador obtiene una copia del proyecto en su equipo mediante Git.
- **Desarrollo de funcionalidades:** cada integrante realiza modificaciones o incorpora nuevas características en su repositorio local.
- **Registro de cambios:** las modificaciones se guardan mediante confirmaciones (commits), conservando el historial de versiones.
- **Publicación de cambios:** los cambios se envían al repositorio remoto para que estén disponibles para los demás integrantes del equipo.
- **Integración de versiones:** los cambios son revisados e incorporados al proyecto principal, manteniendo una única versión actualizada del código.

A continuación, se presenta el funcionamiento general de un repositorio remoto durante el desarrollo colaborativo de software.

Figura 6. Funcionamiento general de un repositorio remoto



Además del almacenamiento del código fuente, GitHub y GitLab incorporan diferentes funcionalidades que apoyan el desarrollo colaborativo de aplicaciones.

Tabla 9. Funcionalidades de GitHub y GitLab.

Funcionalidad	Descripción
Repositorios remotos	Almacenan el código fuente de los proyectos y permiten compartirlo con otros desarrolladores.
Gestión de ramas	Facilitan el desarrollo paralelo de nuevas funcionalidades o correcciones sin afectar la versión principal del proyecto.
Control de versiones	Conservan el historial de cambios realizados sobre el código fuente.
Revisión de código	Permiten analizar y aprobar modificaciones antes de incorporarlas al proyecto principal.
Trabajo colaborativo	Facilitan la participación de varios desarrolladores mediante mecanismos de sincronización y administración de cambios.
Integración con herramientas DevOps	Permiten conectar el repositorio con herramientas de integración, pruebas y despliegue continuo para automatizar procesos de desarrollo.

Aunque ambas plataformas ofrecen funcionalidades similares, presentan algunas características que pueden adaptarse a diferentes necesidades de desarrollo.

Tabla 10. Comparación general entre GitHub y GitLab.

Aspecto	GitHub	GitLab
Enfoque principal	Desarrollo colaborativo y alojamiento de proyectos de software.	Desarrollo colaborativo con herramientas integradas para DevOps.
Integración con Git	Sí	Sí
Gestión de repositorios	Sí	Sí
Gestión de ramas	Sí	Sí
Integración con procesos CI/CD	Disponible mediante herramientas y servicios integrados.	Incorpora funcionalidades CI/CD de forma nativa.

No se trata de determinar cuál plataforma es mejor, sino de reconocer que ambas permiten gestionar repositorios remotos y apoyar el desarrollo colaborativo de aplicaciones mediante Git.

Ejemplo de utilización de un repositorio remoto

Un equipo desarrolla una aplicación para la gestión de inventarios utilizando un repositorio remoto alojado en GitHub. Cada desarrollador crea una copia local del proyecto y trabaja en nuevas funcionalidades de manera independiente. Una vez finalizadas las modificaciones, registra los cambios mediante Git y los publica en el repositorio remoto. Posteriormente, el equipo revisa las contribuciones y las integra al

proyecto principal, garantizando que todos los integrantes trabajen sobre una versión actualizada del código fuente.

Este ejemplo permite identificar que los repositorios remotos facilitan el trabajo colaborativo, centralizan la gestión del código fuente y permiten mantener un historial organizado de las diferentes versiones del proyecto.

Como complemento al proceso de aprendizaje, se recomienda consultar los recursos oficiales de GitHub y GitLab, donde se encuentran disponibles guías, tutoriales y documentación sobre la creación de repositorios, la colaboración entre desarrolladores y la administración de proyectos mediante Git.

GitHub: <https://github.com/>

Documentación de GitHub: <https://docs.github.com/es>

GitLab: <https://about.gitlab.com/es/>

Documentación de GitLab: <https://docs.gitlab.com/>

5.4. Estrategia de ramificación GitFlow

Durante el desarrollo de una aplicación, es común que varios integrantes del equipo trabajen al mismo tiempo en nuevas funcionalidades, correcciones de errores y mejoras del sistema. Si todas estas modificaciones se realizaran directamente sobre la versión principal del proyecto, aumentaría el riesgo de generar conflictos y afectar el funcionamiento de la aplicación.

Para evitar esta situación, se utilizan estrategias de ramificación que permiten organizar el desarrollo del software de manera controlada. Una de las más utilizadas

es GitFlow, un modelo de trabajo que define diferentes ramas (branches) para desarrollar nuevas funcionalidades, preparar versiones y corregir errores sin comprometer la estabilidad del proyecto.

Cada rama cumple una función específica dentro del proceso de desarrollo, permitiendo que los integrantes del equipo trabajen simultáneamente y que los cambios se integren de forma organizada.

Tabla 11. Principales ramas de la estrategia GitFlow.

Rama	Propósito
main	Contiene la versión estable del proyecto, lista para ser utilizada por los usuarios.
develop	Reúne los cambios que se encuentran en desarrollo antes de incorporarlos a la versión estable.
feature	Se utiliza para desarrollar nuevas funcionalidades de forma independiente.
release	Permite preparar una nueva versión del software antes de su publicación.
hotfix	Se emplea para corregir errores críticos detectados en la versión que se encuentra en producción.

El funcionamiento de GitFlow puede comprenderse mediante un ejemplo sencillo.

Supóngase que un equipo desarrolla una aplicación para la gestión de inventarios. Inicialmente, existe una versión estable del sistema, la cual se encuentra almacenada en la rama main.

Cuando el equipo necesita incorporar una nueva funcionalidad, como el registro de proveedores, no modifica directamente la rama main. En su lugar, crea una rama feature, donde desarrolla y prueba la nueva funcionalidad de manera independiente.

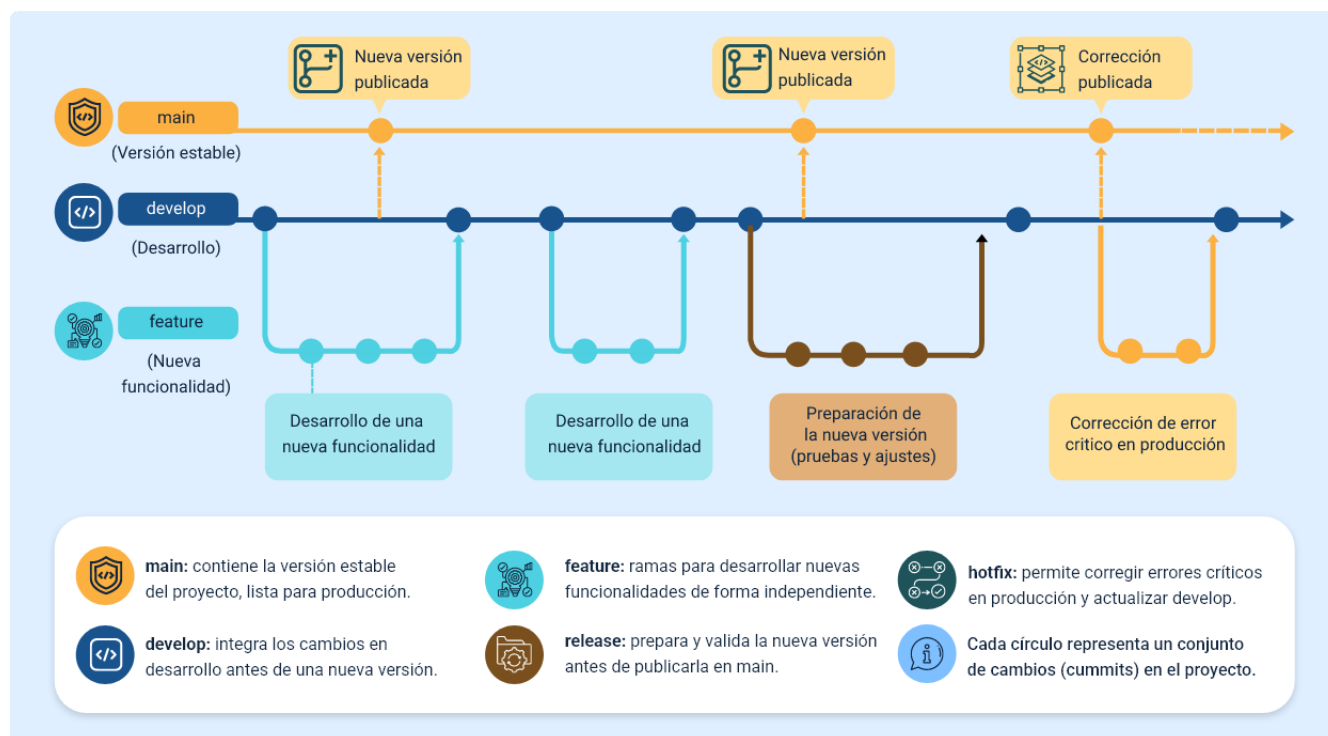
Una vez finalizado el desarrollo, los cambios se integran en la rama develop, que reúne todas las funcionalidades que harán parte de la siguiente versión del sistema.

Cuando el proyecto está listo para publicarse, se crea una rama release, destinada a realizar las pruebas finales y los últimos ajustes antes de liberar la nueva versión.

Finalmente, cuando la versión ha sido validada, esta se incorpora a la rama main, convirtiéndose en la nueva versión estable del proyecto.

Si posteriormente se detecta un error crítico en producción, se crea una rama hotfix a partir de main, se corrige el problema y la solución se incorpora nuevamente tanto a main como a develop, garantizando que la corrección también forme parte de las futuras versiones del software.

Figura 7. Flujo general de la estrategia de ramificación GitFlow



GitFlow organiza el trabajo del equipo mediante ramas con propósitos específicos, permitiendo desarrollar, probar y liberar nuevas versiones de software de manera segura y controlada.

Como complemento al proceso de aprendizaje, se recomienda explorar la plataforma interactiva Learn Git Branching, donde es posible practicar el uso de ramas, fusiones e integración de cambios mediante ejercicios guiados en un entorno simulado.

https://learngitbranching.js.org/?locale=es_ES

5.5. Buenas prácticas en la gestión de código fuente

La gestión del código fuente no solo depende del uso de herramientas como Git o de plataformas para alojar repositorios remotos. También requiere la aplicación de buenas prácticas que favorezcan la organización del proyecto, faciliten el trabajo colaborativo y contribuyan a mantener la calidad del software durante todo su ciclo de desarrollo.

La adopción de estas prácticas permite reducir errores, mejorar la trazabilidad de los cambios y facilitar la integración del trabajo realizado por los diferentes integrantes del equipo de desarrollo.

Tabla 12. Buenas prácticas en la gestión del código fuente.

Buena práctica	Descripción
Realizar commits frecuentes	Registrar los cambios de manera periódica para mantener un historial organizado y facilitar la recuperación de versiones.
Escribir mensajes descriptivos	Utilizar mensajes claros que indiquen el propósito de cada cambio realizado en el código fuente.
Trabajar en ramas independientes	Desarrollar nuevas funcionalidades o corregir errores en ramas específicas antes de integrarlas al proyecto principal.
Mantener el repositorio sincronizado	Actualizar periódicamente el repositorio local con los cambios realizados por otros integrantes del equipo.
Revisar el código antes de integrarlo	Verificar que las modificaciones cumplan con los requisitos del proyecto antes de incorporarlas a la rama principal.

Buena práctica	Descripción
Evitar almacenar información sensible	No incluir contraseñas, claves de acceso o información confidencial dentro del código fuente o del repositorio.
Documentar los cambios importantes	Registrar las modificaciones relevantes para facilitar el mantenimiento y la comprensión del proyecto.

La aplicación de estas buenas prácticas favorece el desarrollo colaborativo, fortalece la calidad del software y facilita la administración del código fuente durante todas las etapas del proyecto.

Ejemplo de aplicación

Un equipo de desarrollo implementa una nueva funcionalidad para una aplicación web. Cada desarrollador trabaja en una rama independiente, registra sus cambios mediante commits con mensajes descriptivos y sincroniza periódicamente su repositorio local con el repositorio remoto. Antes de integrar la funcionalidad al proyecto principal, el equipo revisa el código para verificar su correcto funcionamiento y garantizar que cumple con los estándares definidos para el proyecto.

Este ejemplo permite identificar que la aplicación de buenas prácticas facilita la colaboración entre los integrantes del equipo, mejora la organización del código fuente y contribuye al desarrollo de aplicaciones más confiables.

Como complemento al proceso de aprendizaje, se recomienda consultar la guía oficial de buenas prácticas para el uso de Git disponible en la documentación de

GitHub, donde se presentan recomendaciones para la gestión de repositorios, ramas y colaboración en proyectos de desarrollo de software. <https://docs.github.com/es>

6. Automatización de procesos CI/CD

La automatización constituye uno de los principios fundamentales del enfoque DevOps, ya que permite ejecutar de manera repetitiva y confiable actividades relacionadas con la construcción, validación, prueba e implementación de aplicaciones de software. Mediante la automatización de estos procesos es posible reducir la intervención manual, disminuir la ocurrencia de errores y acelerar la entrega de nuevas versiones del software.

Dentro de este contexto, las herramientas de integración y entrega continua desempeñan un papel fundamental al coordinar las diferentes actividades que conforman el ciclo de desarrollo. Estas herramientas ejecutan automáticamente tareas como la compilación del código, la ejecución de pruebas, la generación de artefactos y el despliegue de aplicaciones, garantizando que cada cambio incorporado al proyecto sea validado antes de su publicación.

Entre las herramientas más utilizadas para este propósito se encuentra Jenkins, una plataforma de automatización de código abierto ampliamente utilizada para implementar procesos de integración continua y entrega continua. Su capacidad para integrarse con diferentes herramientas de desarrollo permite construir flujos de trabajo automatizados que fortalecen la calidad del software y optimizan el proceso de desarrollo.

6.1. Introducción a Jenkins

La automatización de procesos constituye uno de los principios fundamentales del enfoque DevOps, ya que permite reducir tareas manuales, minimizar errores y

acelerar la entrega de nuevas versiones de software. Para lograr este propósito, se utilizan herramientas especializadas que ejecutan automáticamente actividades como la compilación del código, la ejecución de pruebas y el despliegue de aplicaciones.

Jenkins es una herramienta de automatización de código abierto (open source) ampliamente utilizada para implementar procesos de integración continua (CI) y entrega continua (CD). Su función principal consiste en coordinar y ejecutar automáticamente diferentes tareas del ciclo de desarrollo de software cada vez que se detectan cambios en el código fuente.

Jenkins puede integrarse con sistemas de control de versiones como Git, plataformas de alojamiento de repositorios como GitHub y GitLab, herramientas para la ejecución de pruebas automatizadas, tecnologías de contenedores como Docker y diferentes servicios de despliegue. Gracias a esta capacidad de integración, se convierte en un elemento central dentro de los procesos de automatización implementados en entornos DevOps.

Entre las principales características de Jenkins se encuentran:

- **Automatización de procesos:** ejecuta tareas repetitivas relacionadas con la construcción, las pruebas y el despliegue de aplicaciones.
- **Integración con múltiples herramientas:** permite trabajar con plataformas y tecnologías utilizadas durante el desarrollo de software.
- **Extensibilidad mediante complementos:** incorpora funcionalidades adicionales a través de una amplia colección de plugins.
- **Ejecución de tareas programadas o automáticas:** inicia procesos a partir de cambios en el código fuente o de eventos previamente definidos.

- **Monitoreo de procesos:** registra el estado de las ejecuciones y facilita el seguimiento de los resultados obtenidos.

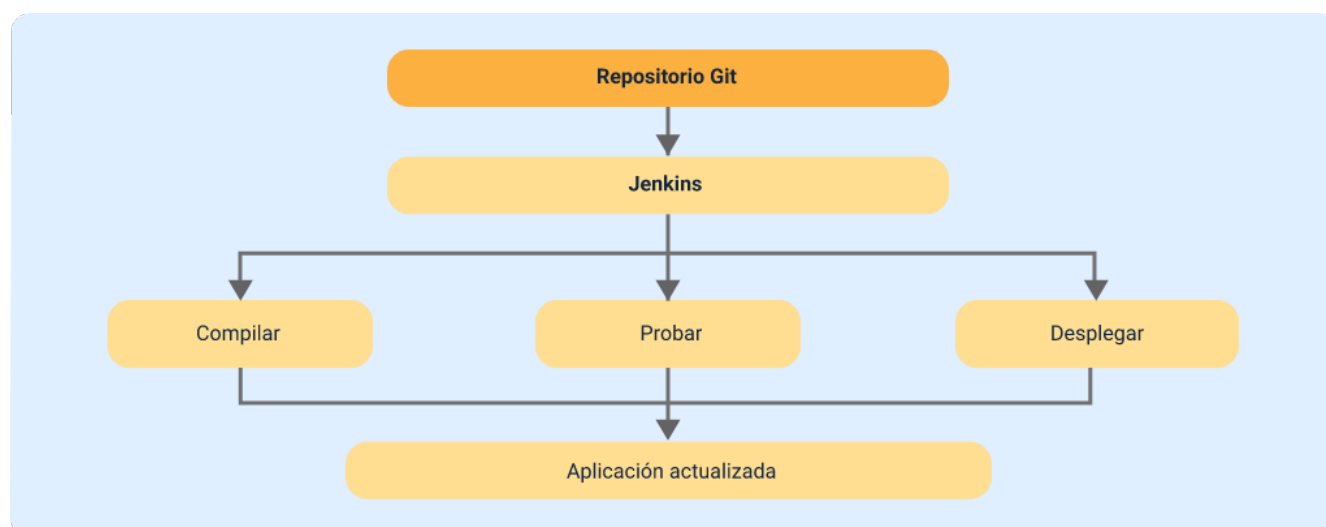
La utilización de Jenkins contribuye a mejorar la calidad del software, reducir los tiempos de entrega y fortalecer la automatización de los procesos de integración y despliegue continuo.

Ejemplo de utilización de Jenkins

Un desarrollador realiza cambios en el código fuente de una aplicación y los publica en el repositorio remoto del proyecto. Jenkins detecta automáticamente la actualización, inicia la compilación del software, ejecuta las pruebas configuradas y, si todas las validaciones son satisfactorias, deja preparada la nueva versión para su implementación en el entorno correspondiente.

Este ejemplo permite identificar que Jenkins automatiza diferentes actividades del ciclo de desarrollo de software, reduciendo la intervención manual y favoreciendo la integración y la entrega continua de aplicaciones.

Figura 8. Jenkins como herramienta de automatización en un flujo DevOps



Jenkins desempeña un papel clave dentro de un flujo DevOps, automatizando la compilación, las pruebas y el despliegue de aplicaciones a partir de los cambios realizados en un repositorio de código fuente.

Como complemento al proceso de aprendizaje, se recomienda consultar la documentación oficial de Jenkins, donde se encuentran disponibles guías, tutoriales y ejemplos relacionados con la automatización de procesos de integración y entrega continua.

Jenkins: <https://www.jenkins.io/>

Documentación de Jenkins: <https://www.jenkins.io/doc/>

6.2. Arquitectura y componentes de Jenkins

Jenkins está diseñado para automatizar procesos relacionados con el desarrollo de software mediante una arquitectura flexible que permite distribuir tareas entre diferentes equipos y entornos de ejecución. Esta organización facilita la ejecución de procesos de integración continua y entrega continua, optimizando el uso de los recursos disponibles y mejorando el rendimiento de las tareas automatizadas.

La arquitectura de Jenkins se basa en un componente principal encargado de administrar el sistema y uno o varios componentes secundarios que ejecutan los procesos definidos por el equipo de desarrollo. Gracias a esta distribución, es posible ejecutar varias tareas de forma simultánea, administrar diferentes proyectos y adaptarse a las necesidades de cada organización.

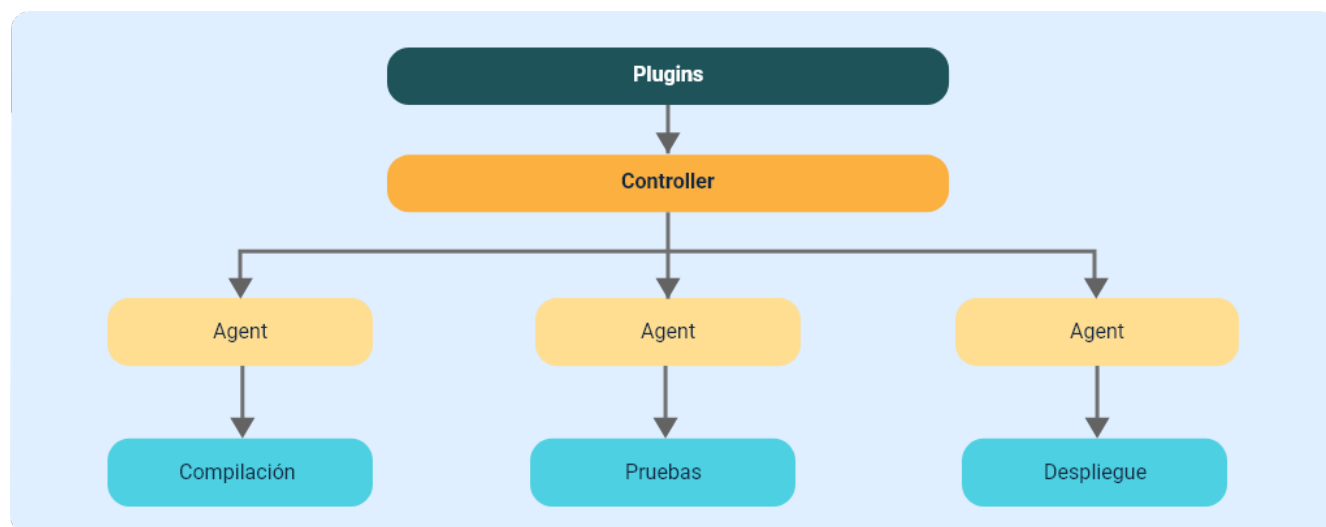
Los principales componentes de Jenkins se describen a continuación.

Tabla 13. Componentes principales de Jenkins

Componente	Descripción
Controller	Es el componente principal de Jenkins. Administra la configuración del sistema, coordina la ejecución de los procesos y distribuye las tareas entre los agentes disponibles.
Agents	Son los equipos o servidores encargados de ejecutar las tareas enviadas por el Controller, como la compilación, las pruebas o el despliegue de aplicaciones.
Jobs	Representan las tareas automatizadas que Jenkins ejecuta de acuerdo con una configuración previamente definida.
Pipelines	Definen el conjunto de etapas que conforman un proceso automatizado, como la construcción, las pruebas y el despliegue del software.
Plugins	Complementos que amplían las funcionalidades de Jenkins e integran la herramienta con otras plataformas y tecnologías utilizadas durante el desarrollo de software.

Cada uno de estos componentes cumple una función específica dentro de la arquitectura de Jenkins y permite automatizar diferentes actividades del ciclo de desarrollo de software de manera organizada y eficiente.

Figura 9. Arquitectura general de Jenkins



La comprensión de estos componentes permite identificar cómo Jenkins distribuye y coordina las diferentes actividades del proceso de integración continua. El siguiente ejemplo describe la forma en que estos elementos interactúan durante la automatización de un proyecto de desarrollo de software.

Ejemplo de funcionamiento

Una organización desarrolla varias aplicaciones de software de manera simultánea. El Controller de Jenkins administra los procesos de automatización y distribuye las tareas entre diferentes Agents. Mientras un agente ejecuta la compilación de una aplicación, otro realiza pruebas automatizadas y un tercero prepara el despliegue de una nueva versión. Gracias a esta distribución, los procesos se ejecutan de manera paralela, optimizando el tiempo y los recursos disponibles.

Este ejemplo permite identificar cómo la arquitectura de Jenkins distribuye las tareas entre sus componentes para automatizar de forma eficiente diferentes actividades del proceso de desarrollo.

6.3. Creación de pipelines de integración continua

Los procesos de integración continua requieren una secuencia organizada de actividades que permitan validar automáticamente los cambios realizados al código fuente antes de incorporarlos a una nueva versión del software. En Jenkins, esta secuencia se implementa mediante los pipelines, los cuales describen las etapas que deben ejecutarse durante el proceso de automatización.

Un pipeline es un flujo de trabajo que organiza y automatiza actividades como la compilación del código, la ejecución de pruebas y la preparación de la aplicación para su implementación. Cada etapa se ejecuta de forma secuencial o paralela, de acuerdo con la configuración definida para el proyecto, permitiendo verificar que el software cumple con los criterios de calidad establecidos.

La creación de un pipeline consiste en definir las diferentes etapas que conforman el proceso de integración continua, especificando las tareas que Jenkins debe ejecutar automáticamente desde que se detecta un cambio en el repositorio hasta que la aplicación queda preparada para su despliegue.

Generalmente, un pipeline de integración continua comprende las siguientes etapas:

- **Obtención del código fuente:** descarga la versión más reciente del proyecto desde el repositorio remoto.

- **Compilación:** construye la aplicación utilizando el código actualizado.
- **Ejecución de pruebas:** verifica el correcto funcionamiento del software mediante pruebas automatizadas.
- **Generación de artefactos:** produce los archivos necesarios para la distribución o implementación de la aplicación.
- **Notificación de resultados:** informa el estado de la ejecución y registra los resultados obtenidos durante el proceso.

La utilización de pipelines permite estandarizar los procesos de integración continua, reducir errores asociados a tareas manuales y garantizar que cada nueva versión del software sea validada antes de continuar con las siguientes etapas del ciclo de desarrollo.

Figura 10. Etapas generales de un pipeline de integración continua



Aunque la configuración de un pipeline puede variar según las necesidades del proyecto, Jenkins utiliza con frecuencia un archivo denominado Jenkinsfile, donde se definen las etapas del proceso de automatización.

Un ejemplo simplificado es el siguiente:

```
pipeline {  
    agent any  
  
    stages {  
        stage('Compilación') {  
            steps {  
                echo 'Compilando la aplicación...'  
            }  
        }  
  
        stage('Pruebas') {  
            steps {  
                echo 'Ejecutando pruebas...'  
            }  
        }  
    }  
}
```

Este ejemplo presenta la estructura básica de un Jenkinsfile, donde se definen dos etapas de un pipeline: la compilación y la ejecución de pruebas. En proyectos reales pueden incorporarse etapas adicionales relacionadas con el análisis del código, la generación de artefactos o el despliegue de la aplicación.

Como complemento al proceso de aprendizaje, se recomienda consultar la documentación oficial de Jenkins sobre la creación de pipelines, donde se presentan ejemplos y buenas prácticas para automatizar procesos de integración continua.

<https://www.jenkins.io/doc/book/pipeline/>

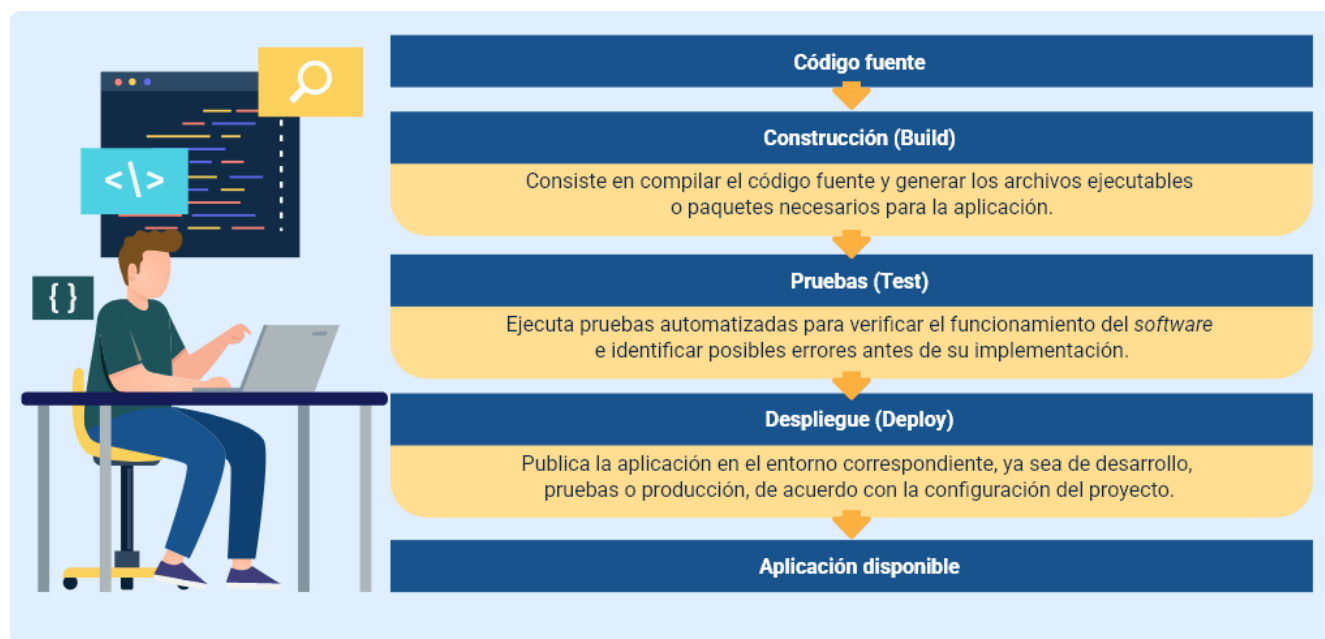
6.4. Automatización de procesos de construcción, pruebas y despliegue

La automatización de procesos permite ejecutar de forma organizada y repetitiva las diferentes actividades necesarias para construir, validar e implementar una aplicación de software. En el enfoque DevOps, esta automatización reduce la intervención manual, disminuye la posibilidad de errores y acelera la entrega de nuevas versiones del producto.

A través de herramientas como Jenkins, las tareas definidas en un pipeline se ejecutan automáticamente cuando se detectan cambios en el código fuente. Esto permite verificar el correcto funcionamiento de la aplicación antes de su publicación, garantizando que cada versión cumpla con los requisitos de calidad establecidos.

Generalmente, un proceso automatizado comprende las siguientes etapas:

Figura 11. Automatización de procesos de construcción, pruebas y despliegue



Automatización de procesos de construcción, pruebas y despliegue

- **Código fuente**
- **Construcción (Build):** consiste en compilar el código fuente y generar los archivos ejecutables o paquetes necesarios para la aplicación.
- **Pruebas (Test):** ejecuta pruebas automatizadas para verificar el funcionamiento del software e identificar posibles errores antes de su implementación.
- **Despliegue (Deploy):** publica la aplicación en el entorno correspondiente, ya sea de desarrollo, pruebas o producción, de acuerdo con la configuración del proyecto.
- **Aplicación disponible**

La automatización de estas actividades permite estandarizar el proceso de desarrollo, mejorar la calidad del software y reducir el tiempo requerido para entregar nuevas versiones a los usuarios.

Ejemplo de automatización

Un desarrollador publica una nueva versión del código fuente en el repositorio del proyecto. Automáticamente, Jenkins inicia el proceso de construcción de la aplicación, ejecuta las pruebas configuradas y, si todas las validaciones son satisfactorias, despliega la nueva versión en el entorno de pruebas. Este proceso se realiza sin intervención manual, reduciendo el tiempo necesario para validar y distribuir el software.

Este ejemplo permite identificar cómo la automatización de procesos fortalece la integración continua, mejora la calidad del software y agiliza la entrega de nuevas versiones. La implementación de estas prácticas genera beneficios que impactan positivamente el desarrollo de software, favoreciendo la eficiencia, la calidad y la confiabilidad de los procesos. A continuación, se presentan algunos de los principales beneficios de la automatización de procesos de construcción, pruebas y despliegue.

Tabla 14. Beneficios de la automatización de procesos de construcción, pruebas y despliegue

Beneficio	Descripción
Reducción de errores	Disminuye las fallas ocasionadas por actividades manuales.
Mayor rapidez	Reduce el tiempo necesario para validar y publicar nuevas versiones.
Calidad del software	Permite ejecutar pruebas de manera consistente en cada integración.

Beneficio	Descripción
Repetibilidad	Garantiza que los procesos se ejecuten siempre bajo las mismas condiciones.
Productividad	Libera al equipo de tareas repetitivas para concentrarse en el desarrollo de nuevas funcionalidades.

6.5. Monitoreo y gestión de procesos CI/CD

La automatización de procesos no finaliza con la ejecución de las actividades de construcción, pruebas o despliegue de una aplicación. Para garantizar la calidad y confiabilidad del desarrollo de software, es necesario realizar un seguimiento permanente a cada una de las tareas ejecutadas dentro de los procesos de integración y entrega continua.

El monitoreo consiste en supervisar el estado de las ejecuciones automatizadas, verificando si las tareas programadas se completan correctamente o si presentan errores que requieran atención. Por su parte, la gestión comprende el análisis de los resultados obtenidos, la administración de las ejecuciones y la implementación de acciones que permitan optimizar el funcionamiento de los procesos CI/CD.

Herramientas como Jenkins proporcionan diferentes mecanismos para realizar el monitoreo y la gestión de los procesos automatizados, permitiendo consultar el estado de las ejecuciones, revisar los registros (logs), identificar fallos y generar notificaciones cuando ocurre algún evento durante la ejecución de un pipeline.

Entre las principales actividades relacionadas con el monitoreo y la gestión de procesos CI/CD se encuentran:

- **Supervisión de ejecuciones:** permite verificar el estado de los procesos automatizados y detectar posibles fallos.
- **Revisión de registros (logs):** facilita el análisis de los eventos generados durante la ejecución de los procesos.
- **Gestión de notificaciones:** informa oportunamente sobre el éxito o el fallo de las tareas automatizadas.
- **Análisis de resultados:** permite identificar oportunidades de mejora y optimizar el desempeño de los procesos.
- **Seguimiento de indicadores:** facilita la evaluación del comportamiento de los procesos automatizados a lo largo del tiempo.

La aplicación de estas actividades contribuye a mantener la estabilidad de los procesos CI/CD y favorece la mejora continua durante el desarrollo de software.

Ejemplo de monitoreo

Durante la ejecución automática de un pipeline, Jenkins detecta que una de las pruebas de integración no se completó correctamente. El equipo de desarrollo consulta los registros de ejecución, identifica la causa del problema y realiza las correcciones necesarias en el código fuente. Posteriormente, el pipeline se ejecuta nuevamente, verificando que las modificaciones solucionaron el inconveniente y permitiendo continuar con el proceso de despliegue.

Este ejemplo permite identificar que el monitoreo y la gestión de procesos CI/CD no solo permiten detectar errores durante la automatización, sino también

implementar acciones de mejora que fortalecen la calidad y confiabilidad del desarrollo de software.

La implementación de actividades de monitoreo y gestión aporta beneficios que permiten mantener la estabilidad de los procesos automatizados y mejorar continuamente su desempeño.

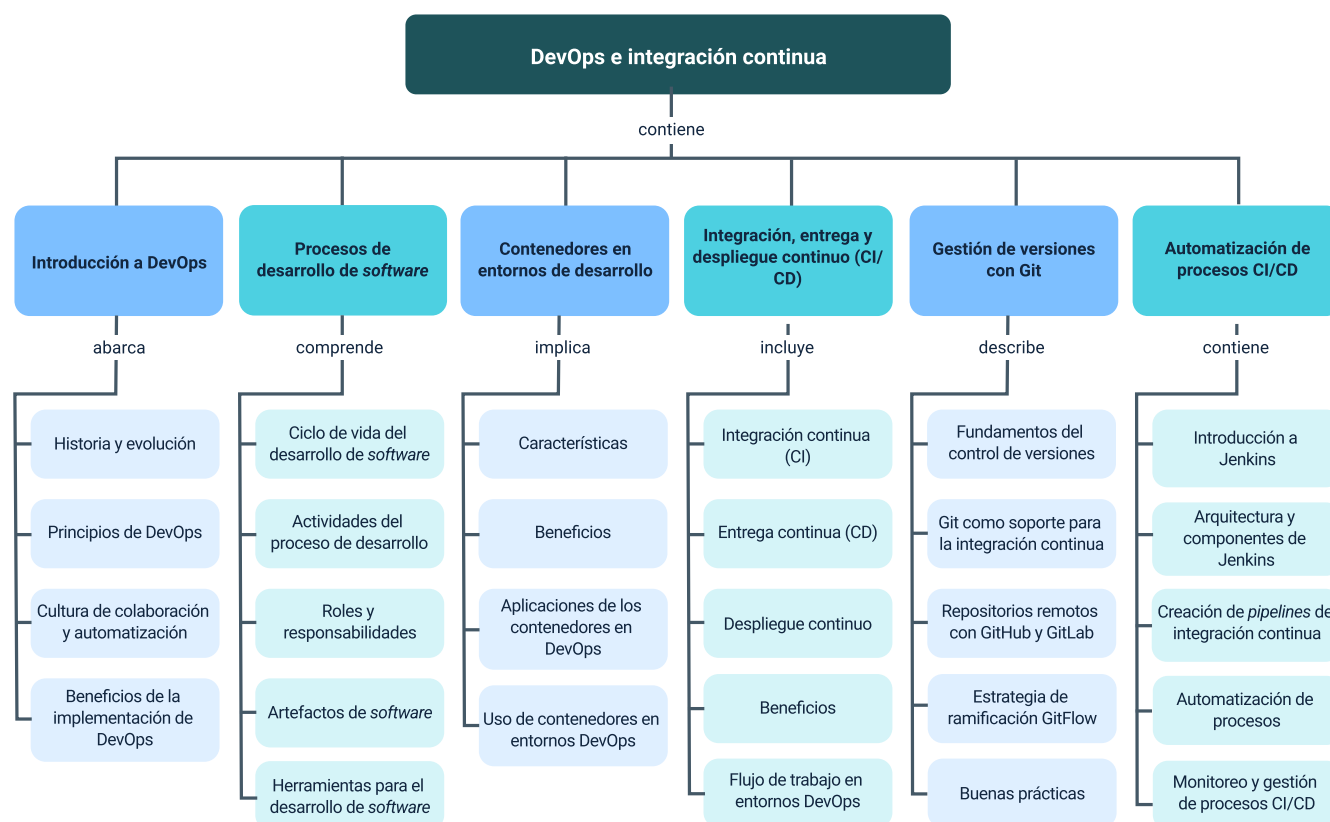
Tabla 15. Actividades de monitoreo y gestión en procesos CI/CD

Actividad	Propósito
Supervisar ejecuciones	Verificar el estado de los procesos automatizados e identificar posibles fallos durante su ejecución.
Revisar registros (logs)	Analizar la información generada por cada ejecución para determinar el origen de posibles errores.
Gestionar notificaciones	Informar oportunamente al equipo sobre el éxito o el fallo de las ejecuciones automatizadas.
Evaluar indicadores	Analizar el comportamiento de los procesos mediante métricas relacionadas con tiempos de ejecución, estabilidad y frecuencia de errores.
Implementar acciones de mejora	Aplicar ajustes que permitan optimizar el rendimiento y la confiabilidad de los procesos CI/CD.

La aplicación de estas actividades permite fortalecer la confiabilidad de los procesos automatizados, facilitar la detección temprana de errores y promover la mejora continua durante el desarrollo de software.

Síntesis

La implementación del enfoque DevOps transforma la forma en que se desarrollan, prueban y despliegan las aplicaciones de software, promoviendo la colaboración entre los equipos de trabajo y la automatización de los procesos. La integración de prácticas como el uso de contenedores, la gestión de versiones, la integración y entrega continua, junto con herramientas como Git y Jenkins, contribuye a optimizar el ciclo de vida del desarrollo de software, fortalecer la calidad de las aplicaciones y agilizar la entrega de soluciones tecnológicas de manera eficiente y confiable.



Glosario

Artefacto de software: documento, archivo o producto generado durante el ciclo de vida del desarrollo de software, como requisitos, modelos, código fuente, ejecutables, manuales o reportes de pruebas.

Automatización: ejecución de tareas o procesos mediante herramientas y tecnologías, reduciendo la intervención manual y aumentando la eficiencia, la repetibilidad y la confiabilidad de las actividades.

CI/CD: conjunto de prácticas que integran la integración continua (Continuous Integration), la entrega continua (Continuous Delivery) y el despliegue continuo (Continuous Deployment) para automatizar el desarrollo y la distribución de software.

Contenedor: tecnología que permite empaquetar una aplicación junto con sus dependencias y configuraciones para ejecutarla de forma consistente en diferentes entornos.

Despliegue continuo (Continuous Deployment): práctica que automatiza la publicación de nuevas versiones del software en un entorno de ejecución una vez superadas las validaciones establecidas.

DevOps: enfoque que integra los equipos de desarrollo y operaciones mediante la colaboración, la automatización y la mejora continua para optimizar el ciclo de vida del desarrollo de software.

Git: sistema de control de versiones distribuido que permite registrar, gestionar y controlar los cambios realizados en el código fuente de un proyecto.

GitFlow: estrategia de ramificación para Git que organiza el desarrollo de software mediante ramas destinadas al desarrollo de funcionalidades, preparación de versiones y corrección de errores.

Integración continua (Continuous Integration): práctica que consiste en incorporar de forma frecuente los cambios realizados al código fuente y validarlos mediante procesos automatizados de construcción y pruebas.

Jenkins: herramienta de automatización de código abierto utilizada para implementar procesos de integración, entrega y despliegue continuo mediante la ejecución de pipelines.

Repositorio remoto: espacio de almacenamiento compartido donde se alojan los proyectos de software y su historial de versiones, facilitando el trabajo colaborativo entre los equipos de desarrollo.

Versionamiento: proceso de registrar, controlar y administrar las diferentes versiones de un proyecto de software, permitiendo realizar seguimiento a los cambios y recuperar versiones anteriores cuando sea necesario.

Referencias bibliográficas

Atlassian. (s. f.). *Git tutorials and training*.

<https://www.atlassian.com/git/tutorials>

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. [https://git-](https://git-scm.com/book/es/v2)

[scm.com/book/es/v2](https://git-scm.com/book/es/v2)

Docker, Inc. (s. f.). *Docker documentation*. <https://docs.docker.com/>

Git. (s. f.). *Git*. <https://git-scm.com/>

GitHub, Inc. (s. f.). *GitHub*. <https://github.com/>

GitHub, Inc. (s. f.). *GitHub Docs*. <https://docs.github.com/es>

GitLab B.V. (s. f.). *GitLab*. <https://about.gitlab.com/es/>

GitLab B.V. (s. f.). *GitLab documentation*. <https://docs.gitlab.com/>

Jenkins. (s. f.). *Documentation*. <https://www.jenkins.io/doc/>

Jenkins. (s. f.). *Jenkins*. <https://www.jenkins.io/>

Jenkins. (s. f.). *Pipeline*. <https://www.jenkins.io/doc/book/pipeline/>

Learn Git Branching. (s. f.). *Learn Git Branching*.

https://learngitbranching.js.org/?locale=es_ES

Red Hat. (s. f.). *What is DevOps?* <https://www.redhat.com/en/topics/devops>

Skoulikari, A. (2023). *Learning Git: A hands-on and visual guide to the basics of Git*. O'Reilly Media.

The Linux Foundation. (s. f.). *What is DevOps?*

<https://training.linuxfoundation.org/>

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico – Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Viviana Esperanza Herrera Quiñonez	Evaluada instruccional	Centro de Comercio y Servicios - Regional Tolima
Jose Yobani Penagos Mora	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Sebastian Trujillo Afanador	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Gilberto Junior Rodríguez Rodríguez	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
María Fernanda Pineda Mora	Evaluada de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
Jorge Bustos Gómez	Validador y vinculator de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima